ĐẠI HỌC QUỐC GIA HÀ NỘI

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

-----ꕥ-----

BÁO CÁO BÀI TẬP LỚN

MÔN: NHẬP MÔN CƠ ĐIỆN TỬ

ĐỀ TÀI: ROBOT DÒ LINE TỰ HÀNH

Giảng viên hướng dẫn: TS. Đỗ Trần Thắng

Mã học phần: 2526I_EMA2027_45

Sinh viên thực hiện:

STT	Tên sinh viên	Lớp	Mã sinh viên
1	Bùi Trọng Kiên	K68M-MT1	23021286
2	Chu Quang Tùng	K68M-MT1	23021350
3	Trương Hồng Tân	K68M-MT1	23021340
4	Nguyễn Quang Thạo	K68M-MT1	23021358
5	Lê Hoàng Khoa	K68M-MT1	23021298

Hà Nội, 12/2025

MỤC LỤC

1. GIỚI THIỆU TỔNG QUAN VỀ ROBOT DÒ LINE TỰ HÀNH (Ý TƯỞNG) 4

1.1. Lý do chọn đề tài 4

1.2. Ý tưởng thiết kế chức năng 5

2. THÀNH PHẦN CHO CHỨC NĂNG 5

2.1. Giới thiệu về linh kiện sử dụng 5

2.1.1. Vi điều khiển Arduino Nano 6

2.1.2. Cảm biến dò line 6

2.1.3. Motor driver TB6612FNG 8

2.1.4. Module hạ áp BUCK DC-DC LM2596 3A 9

2.1.5. Motor N20 và bánh xe (Cơ cấu chấp hành) 10

2.1.6. Pin 11

2.2. Giới thiệu về phần mềm sử dụng 11

2.2.1. Phần mềm Arduino IDE 11

2.2.2. Phần mềm Proteus 11

2.2.3. Phần mềm Visual Studio Code 12

3. CHỨC NĂNG 12

3.1. Nhận biết đường line 12

3.2. Di chuyển bám theo đường line 13

4. PHÂN TÍCH THIẾT KẾ HỆ THỐNG 14

4.1. Thiết kế phần cứng hệ thống 14

4.1.1. Sơ đồ khối và chức năng các khối 14

4.1.2. Thiết kế phần cứng theo chức năng 19

4.1.3. Thiết kế sơ đồ nguyên lý trên Proteus 21
4.1.4. Sơ đồ đấu nối phần cứng 22
4.1.5. Thiết kế thực tế 23
4.2. Sơ đồ phân cấp và sơ đồ theo chức năng của phần mềm 24
4.2.1. Lớp cấp thấp 24
4.2.2. Lớp cấp cao 25
5. THIẾT KẾ PHẦN MỀM 30
5.1. Khai báo cấu hình và biến số toàn cục 31
5.1.1. Thiết lập khai báo kết nối phần cứng (Pin config) 31
5.1.2. Thiết lập khai báo các biến của khối đọc cảm biến `read_black_line()` 32
5.1.3. Khai báo các biến của khối điều khiển `PID_control(), turn()` 33
5.2. Hàm cấu hình khởi tạo `setup()` 34
5.2.1. Khởi tạo giao tiếp và cổng Serial 35
5.2.2. Cấu hình chế độ chân cho cảm biến và nút bấm 35
5.2.3. Cấu hình chân điều khiển động cơ và Driver 36
5.2.4. Kích hoạt trạng thái ban đầu của hệ thống 36
5.3. Vòng lặp chính `loop()` 37
5.3.1. Chế độ vận hành (`BUTTON1`) 38
5.3.2. Chế độ kiểm tra dữ liệu cảm biến (BUTTON2) 39
5.4. Hàm xử lý tín hiệu cảm biến `read_black_line()` 40
5.4.1. Khởi tạo và làm sạch dữ liệu chu kỳ mới 41
5.4.2. Đọc tín hiệu tương tự và chuyển đổi nhị phân 41
5.4.3. Tính toán tổng tích lũy trọng số 43

5.4.4. Lưu các giá trị trong mảng `sensorD[i]` vào các giá trị tương ứng 45

5.5. Hàm điều khiển động cơ `motor()` 45

5.5.1. Giới hạn và chuẩn hóa tín hiệu đầu vào 46

5.5.2. Điều khiển hướng quay động cơ bên trái 46

5.5.3. Điều khiển hướng quay động cơ bên phải 47

5.5.4. Xuất tín hiệu tốc độ thực tế (PWM) 48

5.6. Hàm dừng động cơ `stop()` 48

5.7. Hàm dừng động cơ `move_forward(unsigned long moveTime)` 48

5.8. Hàm rẽ trái và rẽ phải `turnRight(), turnLeft()` 49

5.9. Hàm điều khiển và cân bằng động cơ `PID_control(float Kp, float Ki, float Kd)` 51

5.9.1. Thuật toán PID và chấp hành cho `motor()` 51

5.9.2. Xử lý ngã rẽ và các trường hợp đặc biệt chữ T, dấu cộng, điểm đích 55

6. MÔ PHỎNG VÀ TỐI ƯU HÓA THUẬT TOÁN TRÊN PYTHON 62

6.1. Lý do xây dựng môi trường mô phỏng 62

6.2. Công dụng và hiệu quả thực tiễn 62

6.3. Kết quả mô phỏng và đối chiếu với xe thật 63

7. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ 64

8. ĐỊNH HƯỚNG PHÁT TRIỂN 65

9. TÀI LIỆU THAM KHẢO 67

1. GIỚI THIỆU TỔNG QUAN VỀ ROBOT DÒ LINE TỰ HÀNH (Ý TƯỞNG)

1.1. Lý do chọn đề tài

Việc nghiên cứu và phát triển các hệ thống điều khiển chuyển động tự động là trọng tâm của cuộc Cách mạng Công nghiệp 4.0, với ứng dụng rộng rãi từ robot công nghiệp đến các phương tiện giao thông thông minh. Xe dò line đóng vai trò là một mô hình thử nghiệm cơ bản nhưng toàn diện, cho phép người nghiên cứu nắm vững các nguyên lý cốt lõi về cảm biến, xử lý tín hiệu và lý thuyết điều khiển tự động.

Hình 1: Xe tự hành trong công nghiệp.

Hình 2: Xe AGV kéo hàng.

Mục tiêu của các dự án xe dò line hiện nay không chỉ dừng lại ở khả năng bám theo đường đi mà còn yêu cầu tính ổn định cao và hiệu suất vận hành tốt. Khi triển khai trên thực tế, hệ thống phải đối mặt với các thách thức kỹ thuật quan trọng.

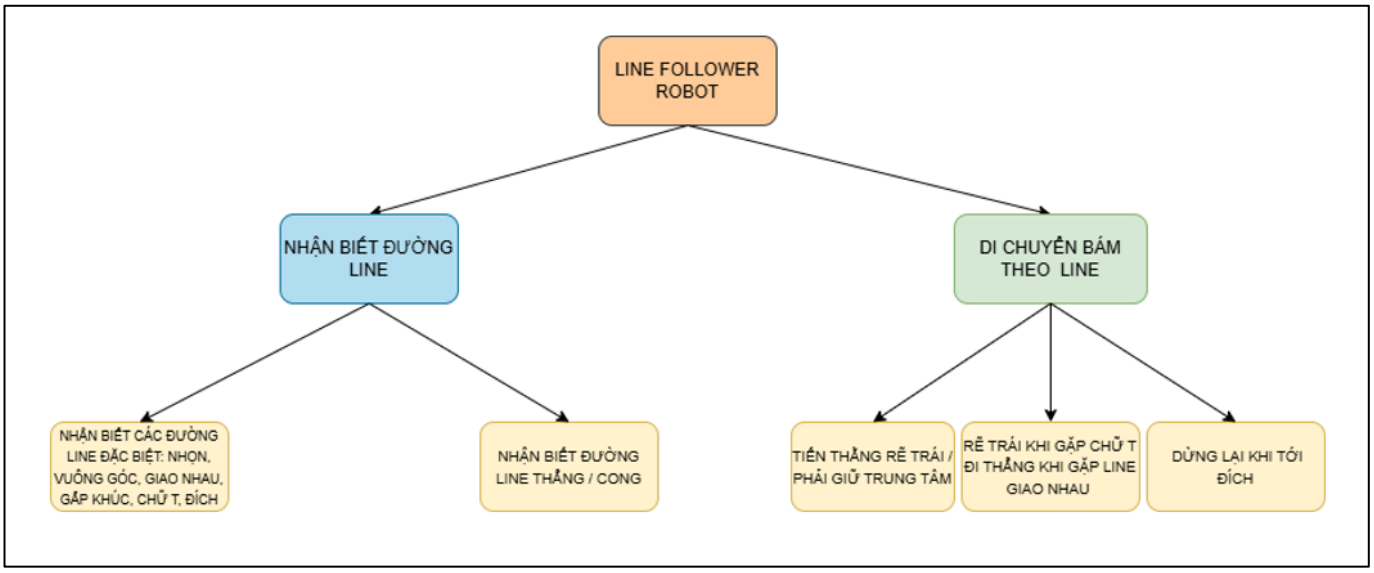
Dự án này được thực hiện nhằm tích hợp và tối ưu hóa sự phối hợp giữa phần cứng và thuật toán điều khiển trên nền tảng Arduino. Mục tiêu cuối cùng là chế tạo một robot dò line có khả năng vận hành tự động với sự cân bằng tốt giữa tốc độ và độ ổn định.

1.2. Ý tưởng thiết kế chức năng

Dựa trên ý tưởng về xe tự hành dò line cho nên nhóm muốn thiết kế xe có hai chức năng là nhận biết đường line và di chuyển bám theo line:

- Chức năng 1: Chức năng nhận biết line giúp xe phát hiện chính xác vị trí đường dẫn, tránh lệch hướng hoặc lạc đường ở những môi trường phức tạp như nhà máy sản xuất,…

- Chức năng 2: Chức năng di chuyển bám line cho phép xe tự động điều khiển động cơ để theo sát đường dẫn mà không cần can thiệp thủ công liên tục, tiết kiệm thời gian và công sức cho người vận hành, đặc biệt hữu ích trong các nhiệm vụ lặp lại trong thời gian dài như vận chuyển hàng hoá tự động,… giúp tăng hiệu quả và năng suất.



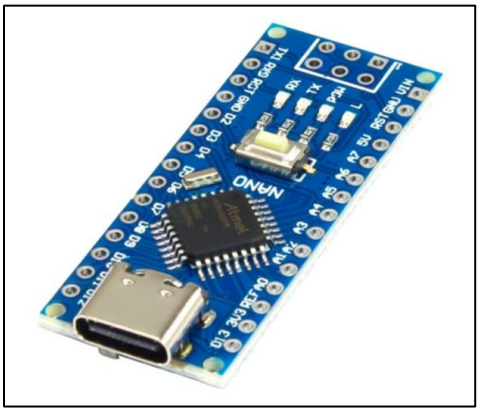
Hình 3: Sơ đồ ý tưởng thiết kế chức năng.

2. THÀNH PHẦN CHO CHỨC NĂNG

2.1. Giới thiệu về linh kiện sử dụng

Các thành phần phần cứng sử dụng trong dự án Robot dò line tự hành:

2.1.1. Vi điều khiển Arduino Nano



Hình 4: Arduino Nano

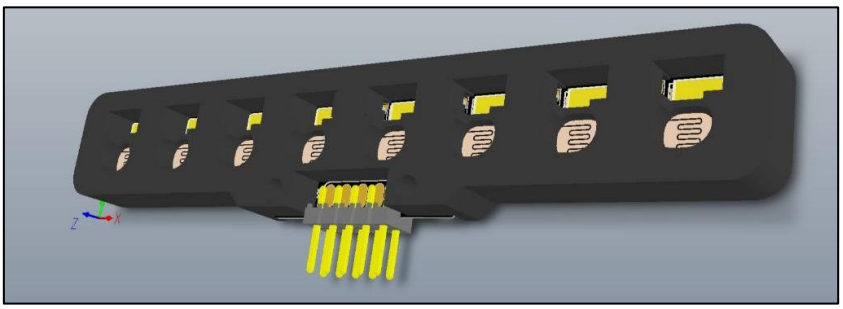
Thông số kỹ thuật:

- Chip xử lý: ATmega328P (8-bit).
- Xung nhịp: 16 MHz.
- Điện áp: Hoạt động 5V; Đầu vào (Vin) 7-12V.
- Bộ nhớ: 32 KB Flash, 2 KB SRAM.
- Cổng giao tiếp: 14 chân Digital (6 chân PWM), 8 chân Analog (ADC 10-bit).

Vai trò trong hệ thống: Đóng vai trò là "Bộ não" trung tâm, thực hiện các nhiệm vụ chính:

- Thu thập và xử lý dữ liệu từ mảng cảm biến hồng ngoại.
- Thực thi thuật toán phản hồi PID để duy trì quỹ đạo.
- Quản lý các logic di chuyển nâng cao như rẽ hướng và điều khiển tốc độ thích nghi.

2.1.2. Cảm biến dò line



Hình 5: Cảm biến dò line

Thông số kỹ thuật:

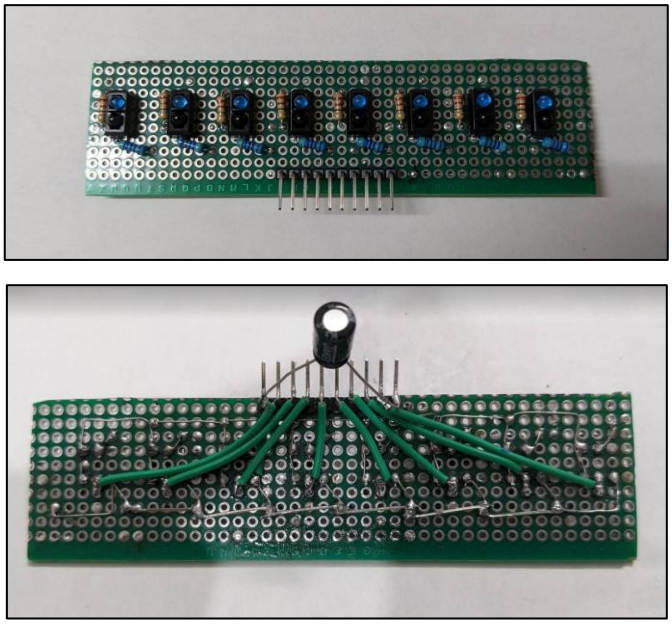
- Điện áp hoạt động: 3.3V đến 5VDC.
- Cấu tạo: Gồm 8 quang trở và 8 LED trắng.

- Khoảng cách phát hiện: 10mm - 15mm.
- Tín hiệu ngõ ra: Dạng số Analog.
- Kích thước: 19 x 96 mm.

Vai trò trong hệ thống:

- **Chức năng:** Đóng vai trò là "đôi mắt" của robot, có nhiệm vụ thu thập dữ liệu về vạch đen trên đường chạy.
- **Xử lý:** Cung cấp thông tin đầu vào để thực hiện phương pháp điều khiển, giúp tính toán sai số liên tục cho bộ điều khiển PID.

– Ban đầu trong quá trình chế tạo và thực nghiệm, nhóm cảm thấy rằng cảm biến dễ xuất hiện trục trặc, thấy cảm biến trả về tín hiệu analog không quá rõ ràng giữa màu đen và màu trắng nên khó chọn ngưỡng nên đã tiến hành tự chế tạo cảm biến bằng điện trở và LED thu/phát hồng ngoại.



Hình 6: Cảm biến dò line nhóm chế tạo

– Cảm biến dò line mới sử dụng 8 bộ thu phát hồng ngoại TCRT5000, điện trở 220Ω nối vào chân dương bóng phát, 4k7Ω nối vào chân dương bóng thu, tụ điện 100µF 35V nối vào 2 chân cấp nguồn.

2.1.3. Motor driver TB6612FNG

Hình 7: Motor driver TB6612FNG

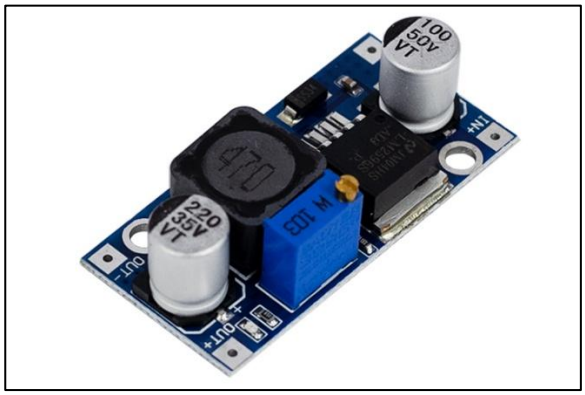
Thông số kỹ thuật:

- Điện áp: VM (nguồn motor) tối đa 15VDC, VCC (nguồn logic) từ 2.7V đến 5.5VDC.
- Dòng điện đầu ra: Trung bình 1.2A mỗi kênh (dòng đỉnh lên đến 3.2A).
- Tần số hoạt động: Hỗ trợ xung PWM lên đến 100KHz.
- Chế độ bảo vệ: Tích hợp bảo vệ quá nhiệt, quá áp và chống ngược nguồn cho motor.
- Kích thước: 20x20 mm.

Vai trò trong hệ thống:

- **Chức năng:** Đóng vai trò là khối chấp hành trung gian, chuyển đổi tín hiệu điều khiển từ Arduino thành năng lượng vật lý để quay động cơ.
- **Ứng dụng:** Thực hiện điều khiển hai động cơ N20, giúp robot thực hiện các chuyển động tiến, lùi, rẽ hướng và duy trì tốc độ thích nghi theo quỹ đạo đường line.

2.1.4. Module hạ áp BUCK DC-DC LM2596 3A



Hình 8: Module hạ áp.

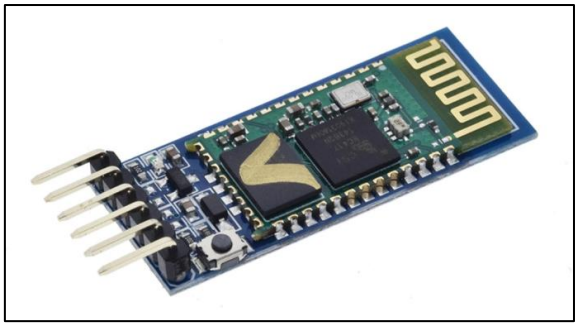
Thông số kỹ thuật:

- Điện áp đầu vào: 7,4 VDC
- Điện áp đầu ra: Được cố định chính xác tại mức 5V.
- Kích thước: 8.6 x 12.33 mm

Vai trò trong hệ thống:

- **Chức năng bảo vệ:** Thực hiện hạ áp từ nguồn pin chính (7.4V) xuống 5V để cấp nguồn an toàn cho Driver TB6612FNG và mảng cảm biến dò line.
- **Độ tin cậy cao:** Đảm bảo hệ thống xử lý không bị nhiễu tín hiệu hoặc hư hỏng do biến động điện áp từ động cơ, duy trì tính chính xác cho các phép toán điều khiển PID.

2.1.5. Module Bluetooth HC05



Hình 9: Module Bluetooth HC05.

Thông số kỹ thuật:

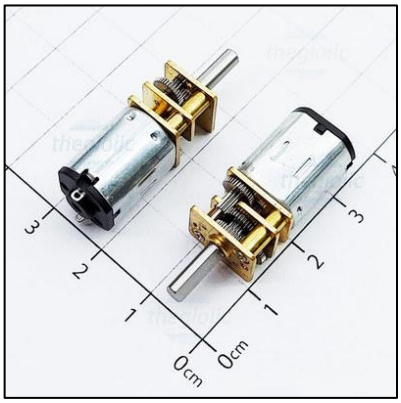
- Điện áp hoạt động: 5VDC 30mA

- Baudrate : 1200, 2400, 4800, 9600, 19200, 38400, 57600, 115200
- Dải tần hoạt động : 2.4GHz
- Kích thước : 26.9mm x 13mm x 2.2mm

Vai trò trong hệ thống:

- Dùng để debug phần mềm, khảo sát xung motor, vị trí xe, ……

2.1.5. Motor N20 và bánh xe (Cơ cấu chấp hành)



Thông số kỹ thuật:

- **Động cơ:** Model GA12-N20, tốc độ quay 500 vòng/phút (rpm), trọng lượng siêu nhẹ chỉ 9g.
- **Trục truyền động:** Đường kính 3mm, dạng trục chữ D chống trượt.
- **Bánh xe:** Đường kính ngoài 43mm, bề rộng 18mm, vỏ bằng cao su giúp tăng độ bám đường.

Hình 10 và 11: Motor N20 và bánh xe

Vai trò trong hệ thống:

- **Chức năng:** Là bộ phận tạo chuyển động chính, thực hiện các lệnh tiến, lùi và rẽ trái/phải từ khối điều khiển.

- **Hiệu suất:** Sự kết hợp giữa động cơ 500rpm và bánh xe 43mm giúp robot đạt được tốc độ tối ưu trên đường đua mà vẫn đảm bảo độ chính xác khi vào cua.

2.1.6. Pin

Hình 12: Pin

Thông số kỹ thuật:

- **Loại pin:** Lithium-ion (Li-ion).
- **Dung lượng:** 1500mAh, đảm bảo thời gian vận hành liên tục cho robot.
- **Điện áp:** 7.4V.
- **Đặc tính:** Dòng xả ổn định, giúp động cơ đạt được công suất tối ưu khi cần tăng tốc.

Vai trò trong hệ thống:

- **Chức năng:** duy trì hoạt động cho tất cả các khối chức năng như vi điều khiển, cảm biến và động cơ.

2.2. Giới thiệu về phần mềm sử dụng

2.2.1. Phần mềm Arduino IDE

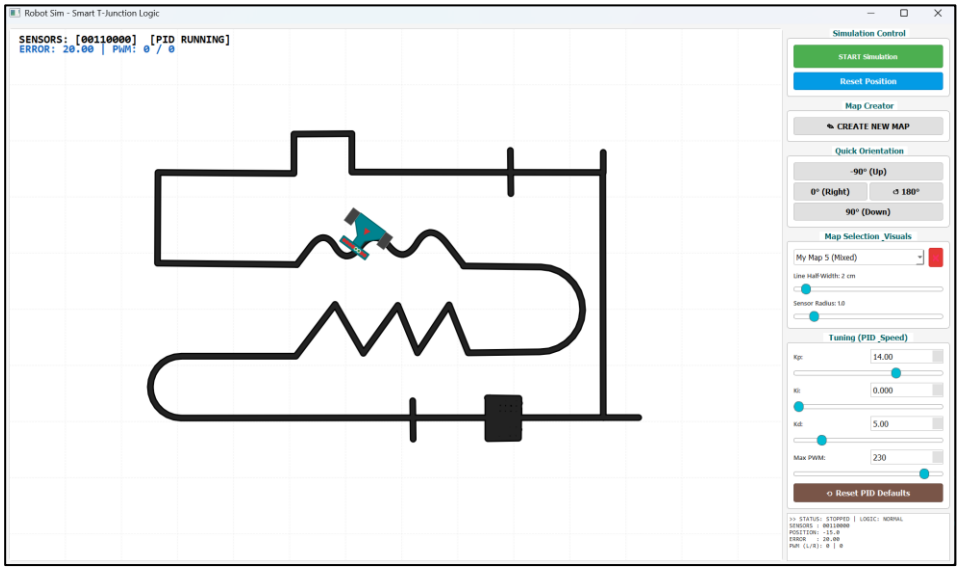
- Sử dụng để nạp code cho vi điều khiển, có khả năng tương thích với nhiều vi điều khiển khác nhau.
- Công cụ phần mềm mã nguồn mở, sử dụng để soạn thảo, biên dịch, debug,… đóng vai trò thiết lập logic điều khiển robot.
- Giao diện thân thiện, thư viện hỗ trợ phong phú và khả năng tương thích với nhiều dòng bo mạch khác nhau.

2.2.2. Phần mềm Proteus

- Mô phỏng mạch điện tử, thiết kế sơ đồ nguyên lý trên máy tính.

2.2.3. Phần mềm Visual Studio Code

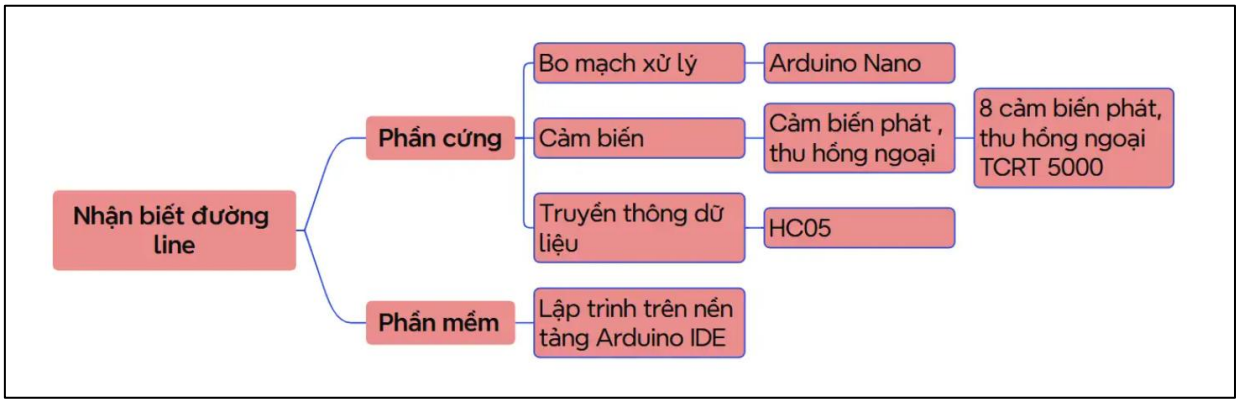
- Sử dụng làm môi trường lập trình chính để xây dựng và vận hành chương trình mô phỏng robot bằng ngôn ngữ Python. Phần mềm mô phỏng giúp kiểm chứng lý thuyết điều khiển trước khi áp dụng lên phần cứng thực tế.
- Trực quan hoá dữ liệu: Hiển thị thời gian thực quỹ đạo di chuyển của robot trên các map giả định khác nhau.



Hình 13: Mô phỏng trên phần mềm VS Code.

3. CHỨC NĂNG

3.1. Nhận biết đường line

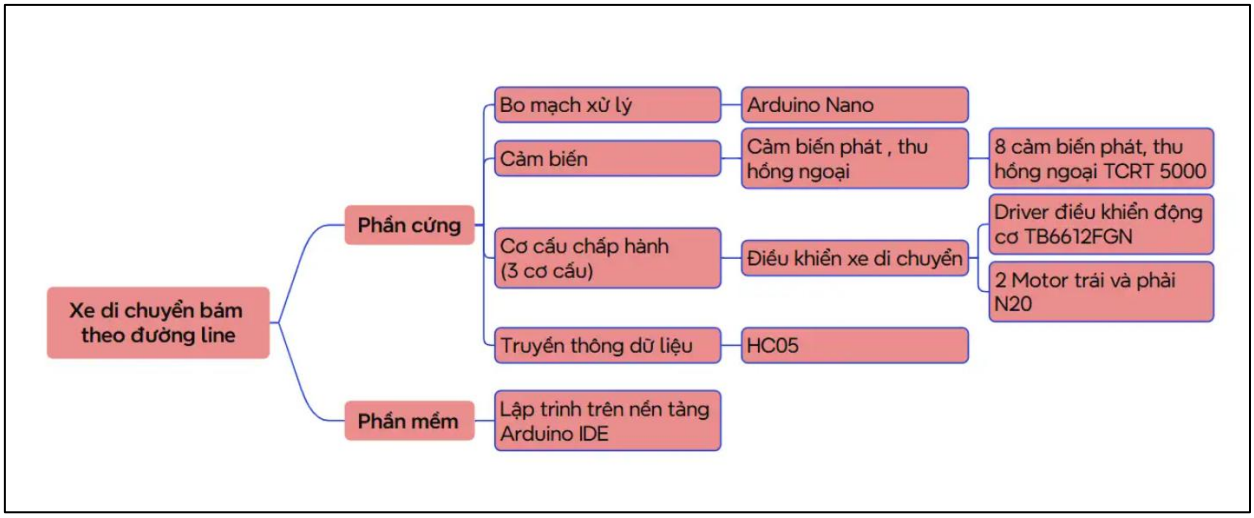


Hình 14: Sơ đồ chức năng nhận biết đường line.

Sơ đồ chia quy trình nhận biết đường line thành hai mảng chính là phần cứng và phần mềm.

- **Thành phần phần cứng:**
    - **Bo mạch xử lý:** Sử dụng Arduino Nano làm trung tâm điều hành, tiếp nhận và xử lý mọi dữ liệu từ cảm biến.
    - **Hệ thống cảm biến:** Gồm mảng 8 cảm biến hồng ngoại TCRT5000, có nhiệm vụ phát và thu tín hiệu để xác định vị trí vạch line trên mặt sân.
    - **Truyền thông:** Module HC05 hỗ trợ gửi/nhận dữ liệu để giám sát trạng thái nhận diện đường đi.
- **Nền tảng phần mềm:** Sử dụng môi trường lập trình Arduino IDE để hiện thực hóa các thuật toán xử lý tín hiệu từ cảm biến.

3.2. Di chuyển bám theo đường line



Hình 15: Sơ đồ chức năng di chuyển bám theo đường line.

Sơ đồ này mở rộng thêm phần Cơ cấu chấp hành để mô tả đầy đủ quy trình từ nhận tin đến vận hành thực tế.

Cơ cấu chấp hành và điều khiển:

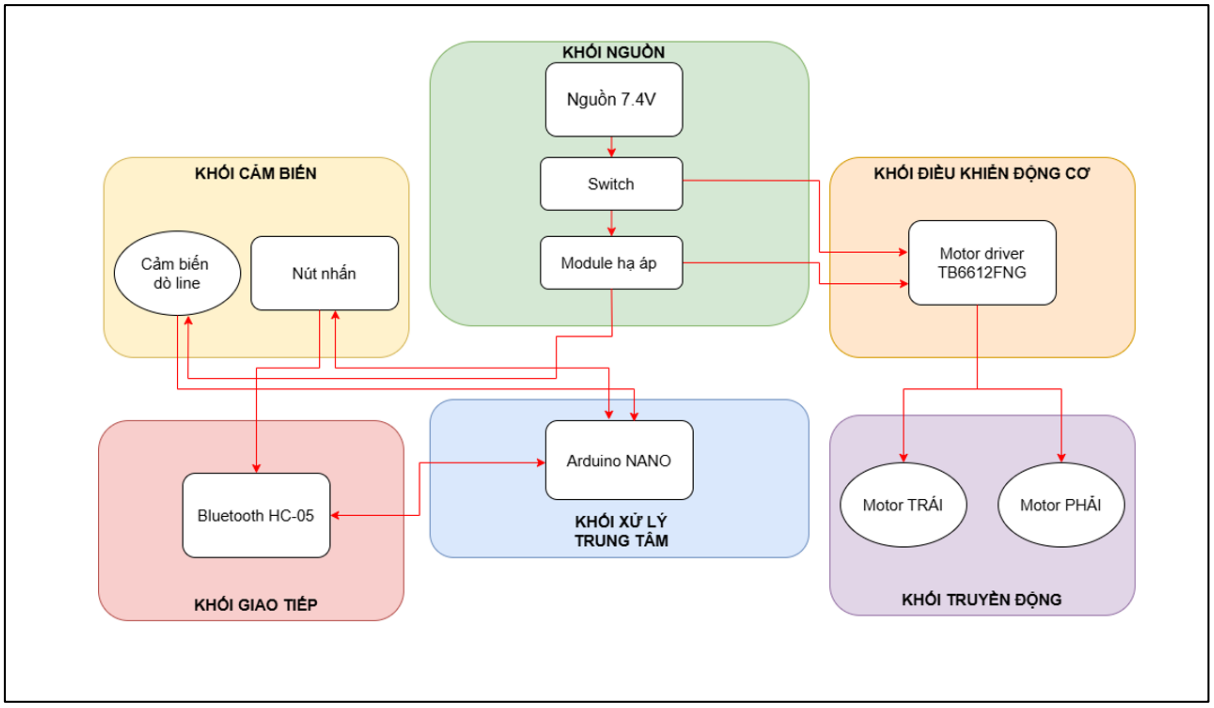
- **Driver điều khiển:** Sử dụng driver TB6612FNG để điều phối năng lượng.
- **Động cơ:** Trang bị 2 Motor N20 (trái và phải) để thực thi lệnh di chuyển.

Phần mềm đóng vai trò cầu nối, nhận dữ liệu từ cảm biến để ra lệnh cho driver điều khiển động cơ bám sát đường đi.

4. PHÂN TÍCH THIẾT KẾ HỆ THỐNG

4.1. Thiết kế phần cứng hệ thống

4.1.1. Sơ đồ khối và chức năng các khối



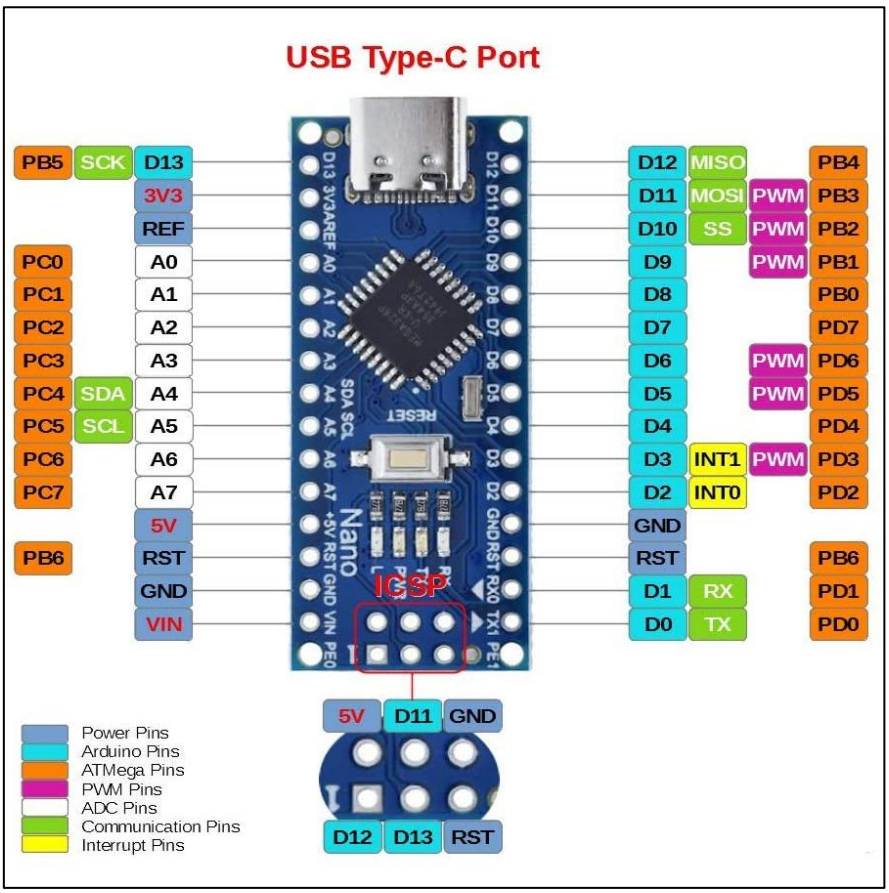
Hình 16: Sơ đồ tổng thể của hệ thống.

- **Khối nguồn:**

  – Nguồn cấp: Sử dụng pin 7.4V cấp cho chân VM của Driver, đầu vào module hạ áp và Vin của vi điều khiển.

– Module hạ áp BUCK DC-DC LM2596 3A: Chuyển đổi từ 7.4V xuống mức 5V để nuôi cảm biến và driver TB6612FNG.

- **Khối xử lý trung tâm:**



Hình 17. Arduino Nano

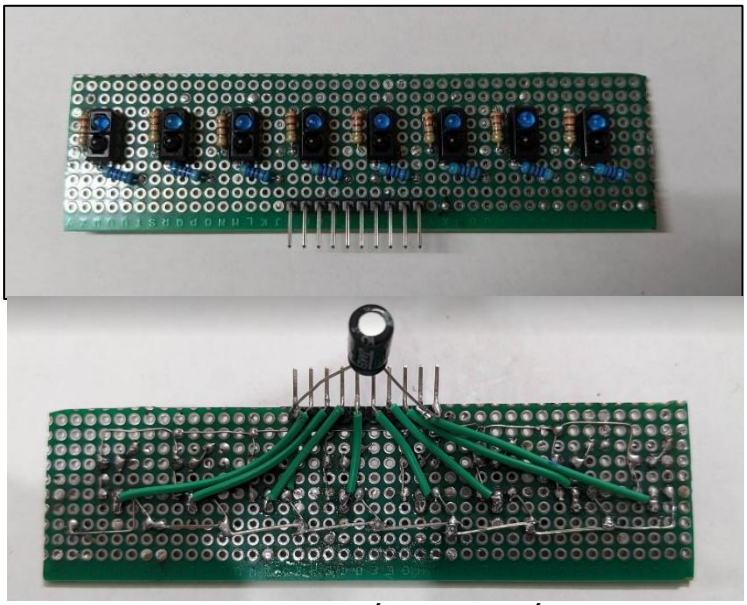
– Xử lý tín hiệu: Nhận dữ liệu Analog từ cảm biến để xác định vị trí vạch kẻ.

– Tính toán PID: Áp dụng thuật toán để tính toán sai số và đưa ra lệnh điều khiển động cơ tối ưu.

– Điều khiển chấp hành: Xuất xung PWM và tín hiệu logic để điều tiết tốc độ, hướng quay của robot qua driver TB6612FNG.

– Quản lý an toàn: Điều khiển chân STBY để kích hoạt hoặc tắt động cơ.

- **Khối cảm biến**



Hình 18: Cảm biến nhóm chế tạo.

– Cấu tạo: Cảm biến dò line sử dụng 8 bộ thu phát hồng ngoại TCRT5000, điện trở 220Ω nối vào chân dương bóng phát, 4k7Ω nối vào chân dương bóng thu, tụ điện 100µF 35V nối vào 2 chân cấp nguồn.

– Kết nối:

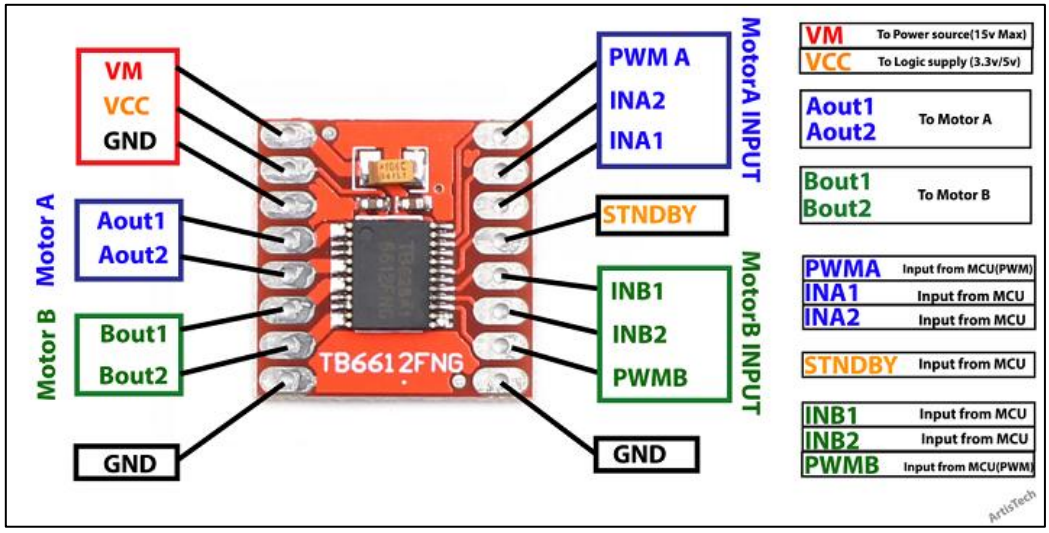
  - Nguồn: Chân VCC nối module hạ áp 5V, chân GND nối chung hệ thống.

  - Tín hiệu: 8 chân (S1-S8) nối trực tiếp vào các chân Analog (A0-A7) của Arduino Nano.

– Nguyên lý: Phân biệt nền trắng và vạch đen dựa trên cường độ ánh sáng phản xạ, sau đó gửi tín hiệu Analog về vi điều khiển để tính toán vị trí của robot.

- **Khối điều khiển động cơ**

Khối này đóng vai trò trung gian, chuyển đổi lệnh logic từ vi điều khiển thành dòng điện công suất để vận hành động cơ.



Hình 19: Driver điều khiển động cơ TB6612FNG

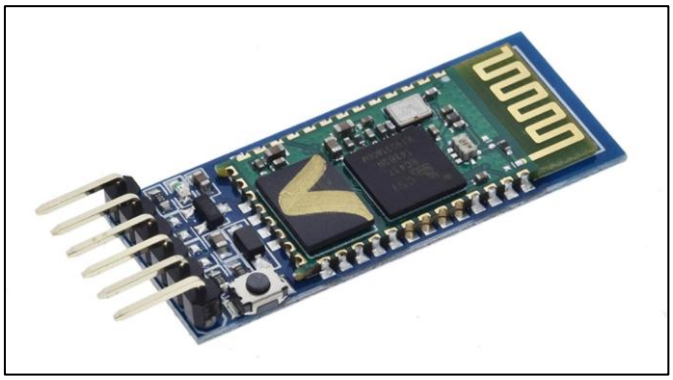
- **Kết nối phần cứng:**
  - Động cơ: Hai động cơ N20 nối vào các đầu ra AOUT và BOUT.
  - Điều khiển: Nhận tín hiệu hướng (IN1, IN2), tốc độ (PWM) và lệnh kích hoạt (STBY) từ vi điều khiển.
  - Nguồn điện: Chân VCC nhận 5V (nuôi mạch logic) và chân VM nhận 7.4V từ pin (cấp công suất cho động cơ).
- **Chức năng:**
  - Điều chỉnh dòng điện để kiểm soát chính xác chiều quay và tốc độ của robot.
  - Tích hợp các cơ chế bảo vệ hệ thống như chống quá dòng, quá nhiệt và chống ngược cực.

- **Khối truyền động**

Hình 20: Động cơ N20.

- **Nhiệm vụ:** Chuyển đổi điện năng thành cơ năng để vận hành bánh xe.
- **Điều hướng:** Hai động cơ hoạt động độc lập, sự chênh lệch tốc độ giữa chúng quyết định khả năng rẽ và bám line của robot.

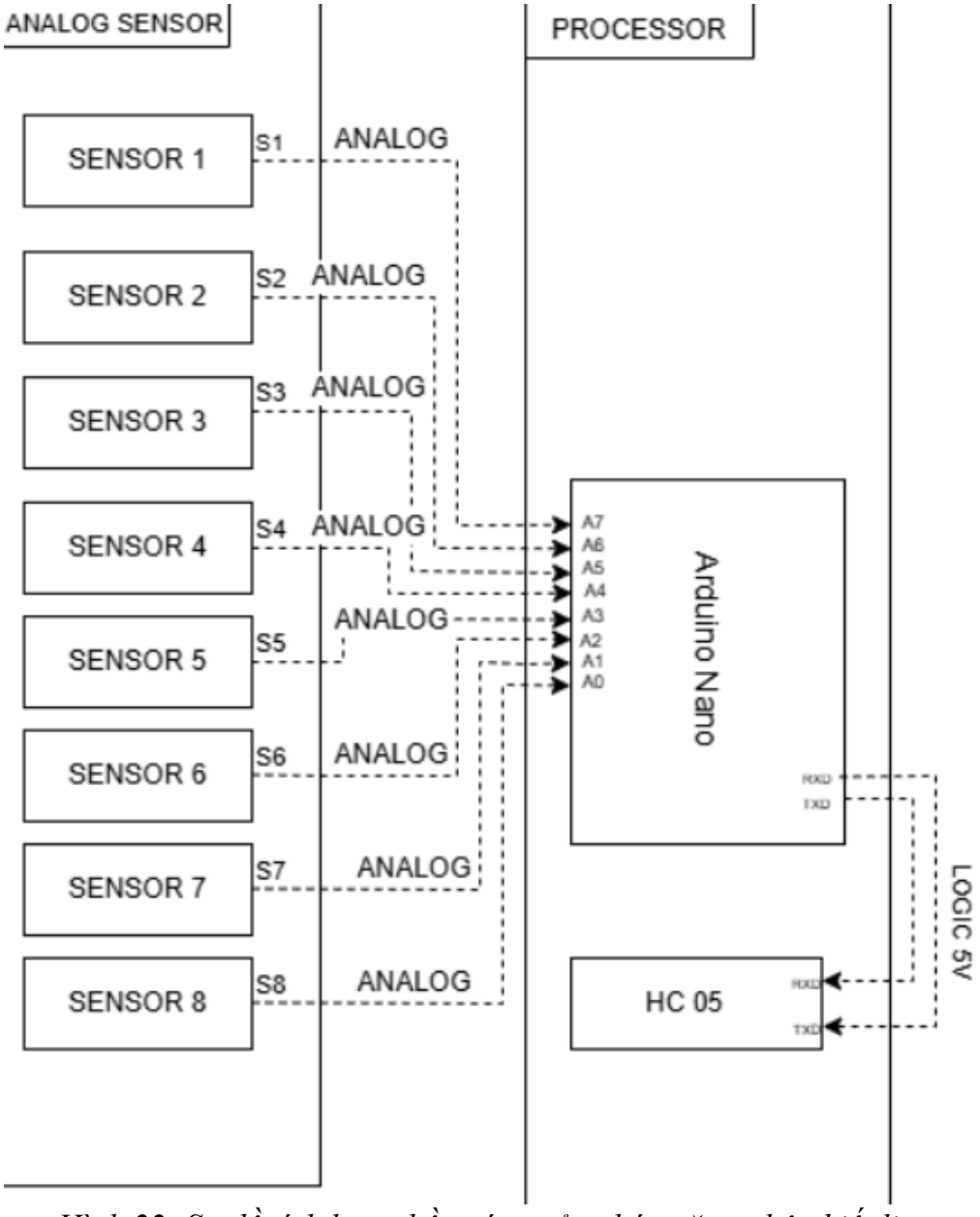
- **Khối giao tiếp**



Hình 21: Module Bluetooth HC05.

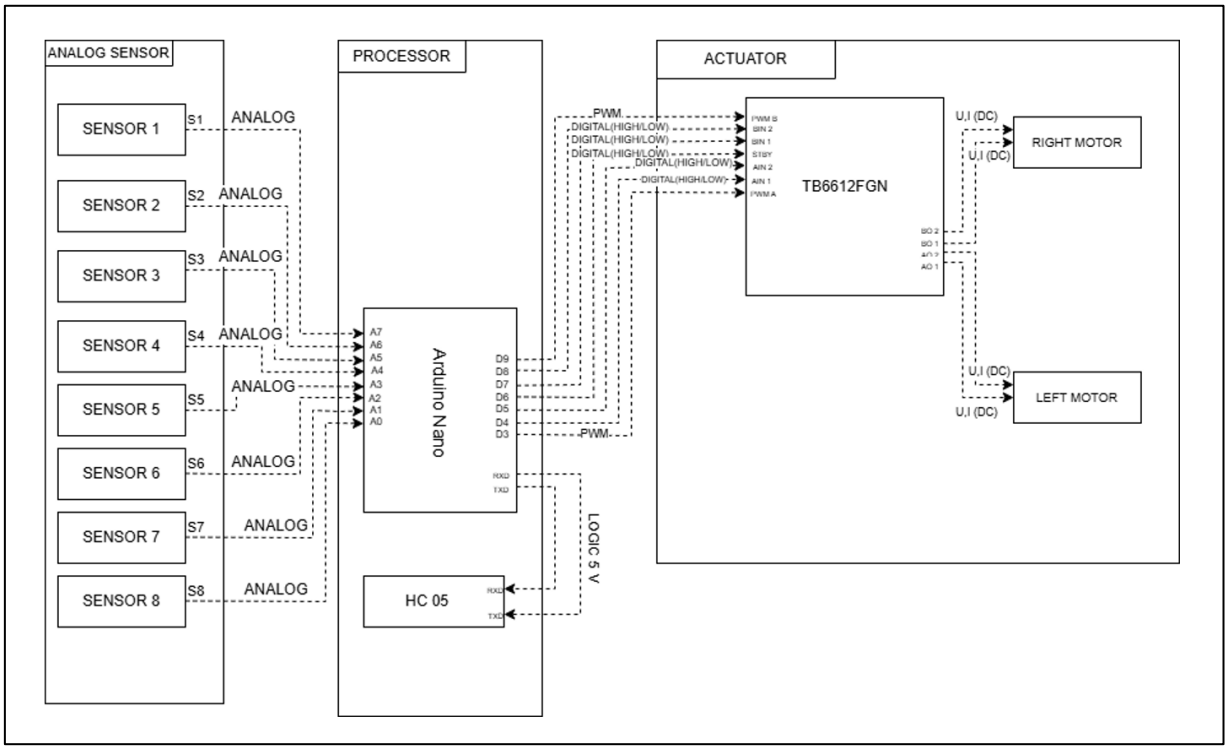
- Kết nối truyền thông không dây tới điện thoại để dễ dàng quan sát, giúp việc debug dễ dàng.

4.1.2. Thiết kế phần cứng theo chức năng



Hình 22: Sơ đồ tích hợp phần cứng của chức năng nhận biết line.

- **Khối đầu vào (Analog Sensor)**:
    - Mảng 8 cảm biến (S1 đến S8) chuyển đổi cường độ ánh sáng phản xạ thành tín hiệu Analog.
    - Các tín hiệu này được đưa vào các chân từ A0 đến A7 của Arduino Nano để xử lý dữ liệu.
- **Khối xử lý (Processor)**:
    - Arduino Nano tiếp nhận dữ liệu analog, thực hiện thuật toán PID để đưa ra quyết định lái.
    - Duy trì giao tiếp hai chiều với module HC05 thông qua các chân TXD/RXD sử dụng mức logic 5V ổn định.
- **Luồng dữ liệu**: Sơ đồ thể hiện tính liên tục từ việc thu thập tín hiệu môi trường đến việc giao tiếp trạng thái ra bên ngoài.

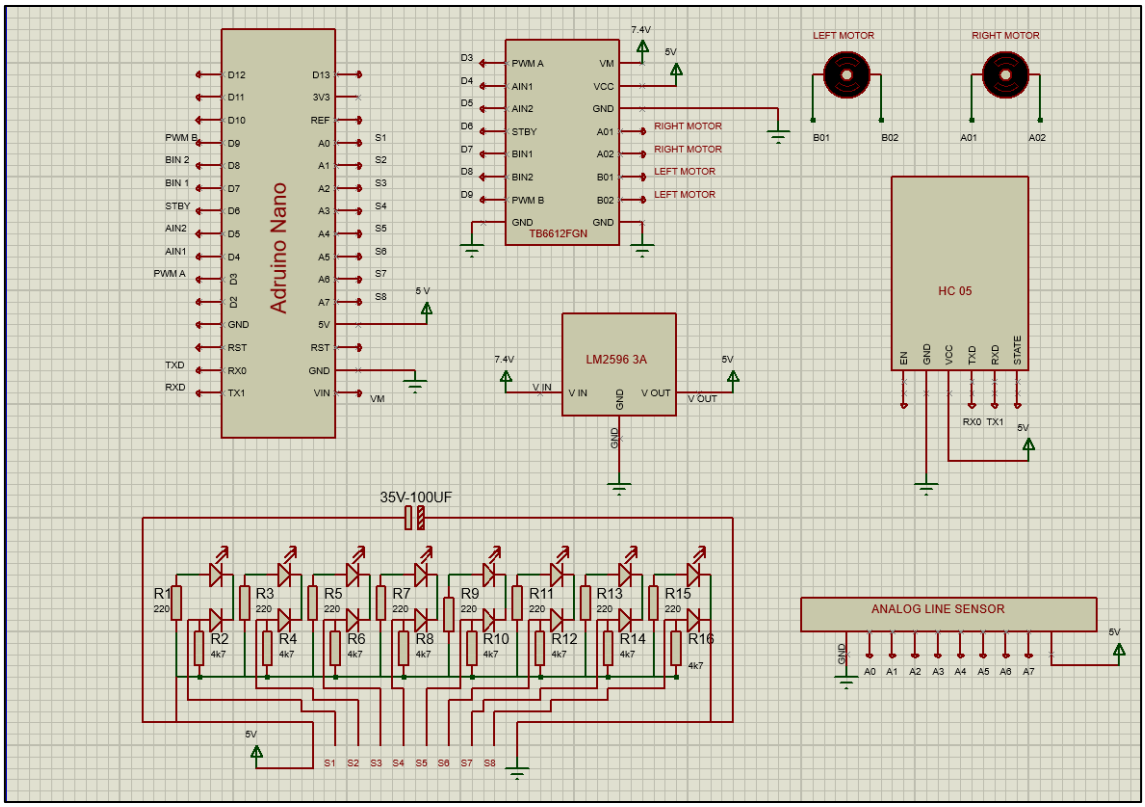


Hình 23: Sơ đồ tích hợp phần cứng của chức năng di chuyển bám line.

Sơ đồ làm rõ mối quan hệ giữa ba khối: Sensor (Thu nhận) -> Processor (Xử lý) -> Actuator (Chấp hành).

- **Kết nối Driver**: Các chân Digital (D3-D9) của Arduino cung cấp xung PWM và tín hiệu logic để điều khiển Driver TB6612FNG.
- **Vận hành động cơ**: Driver nhận lệnh và cấp nguồn cho 2 động cơ N20 (Right/Left Motor), cho phép robot thực hiện các hành động di chuyển phức tạp dựa trên thuật toán bám line.

4.1.3. Thiết kế sơ đồ nguyên lý trên Proteus



Hình 24: Sơ đồ nguyên lý.

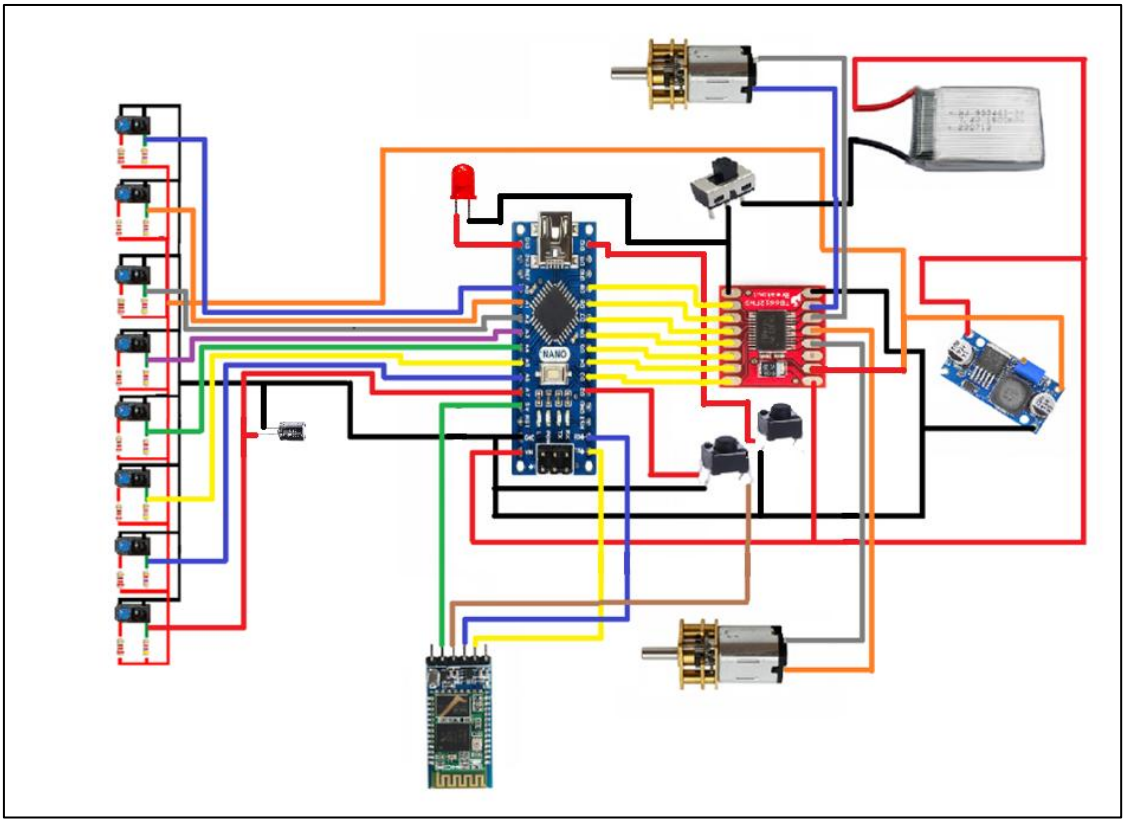
Sơ đồ mạch điện chi tiết thể hiện các kết nối vật lý và quản lý năng lượng:

- **Hệ thống nguồn:** Sử dụng module hạ áp LM2596 3A để chuyển đổi nguồn tổng (7.4V) thành các mức điện áp ổn định (5V) cung cấp cho driver TB6612FNG và cảm biến.

- **Mạch cảm biến:** Mô tả chi tiết cặp hồng ngoại TCRT5000, điện trở hạn dòng và tụ ổn định nguồn.

- **Kết nối Driver:** Thể hiện rõ cách đấu nối các chân điều khiển từ Arduino sang TB6612FNG để điều khiển 2 động cơ N20.

4.1.4. Sơ đồ đấu nối phần cứng



Hình 25: Sơ đồ đấu nối phần cứng.

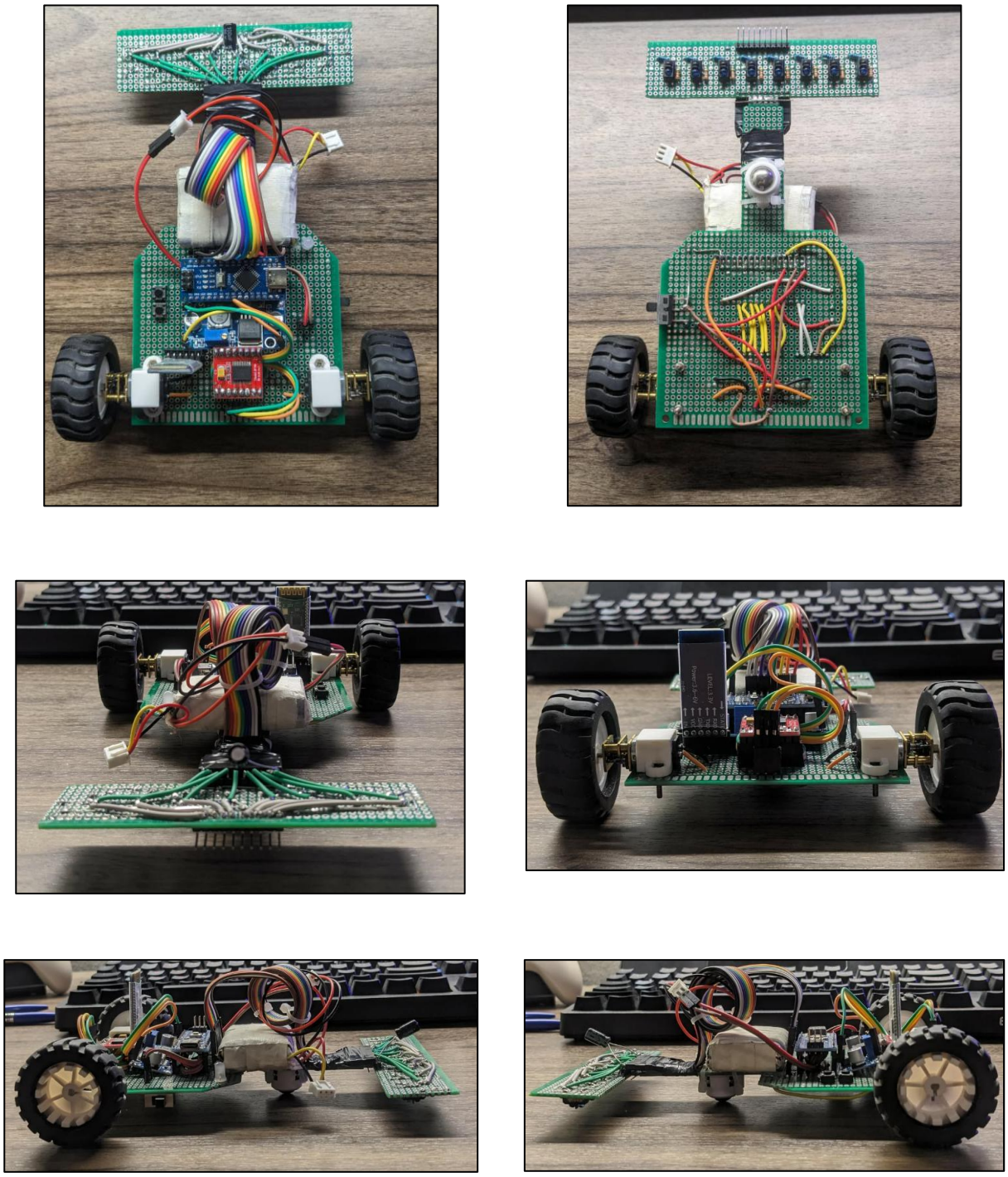
Sơ đồ cung cấp cái nhìn chi tiết về mặt kỹ thuật điện và lắp ráp vật lý:

- Nguồn: Sử dụng pin 7.4V cấp thẳng vào Vin của vi điều khiển và vào Vm của driver điều khiển động cơ. Sử dụng module hạ áp LM2596 3A để chuyển đổi nguồn pin (7.4V) thành các mức điện áp 5V ổn định, cấp nguồn cho cảm biến và driver TB6612FNG. Cấp 5V từ vi điều khiển cho HC05.

- Mạch cảm biến chi tiết: Thể hiện cấu trúc mảng LED hồng ngoại và các điện trở hạn dòng (220Ω, 4.7kΩ) cùng tụ điện lọc nhiễu (100μF 35V).

- Kết nối Driver & Động cơ: Minh họa các chân hướng từ Arduino sang Driver TB6612FNG, cũng như cách kết nối đầu ra Motor tới các động cơ N20.

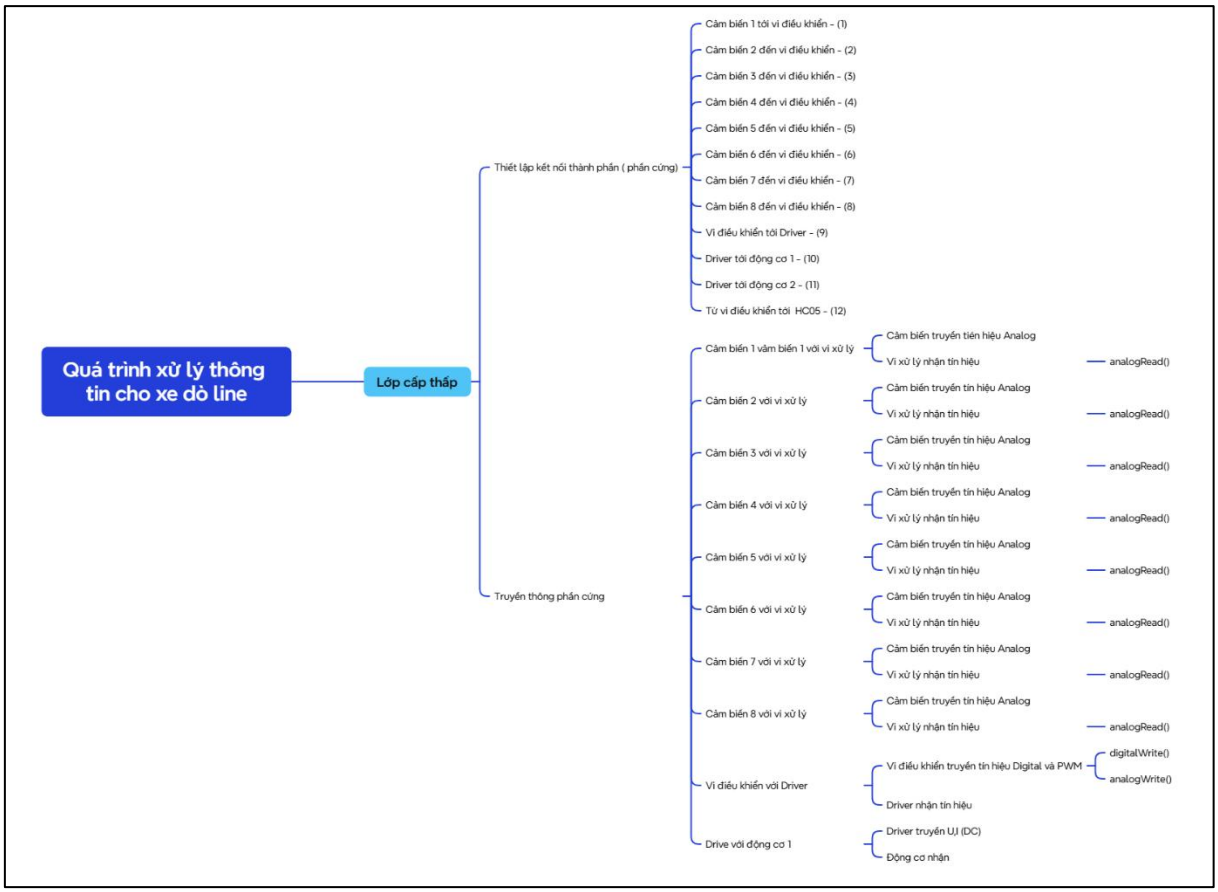
4.1.5. Thiết kế thực tế



Hình 26: Thiết kế thực tế của xe.

4.2. Sơ đồ phân cấp và sơ đồ theo chức năng của phần mềm

4.2.1. Lớp cấp thấp



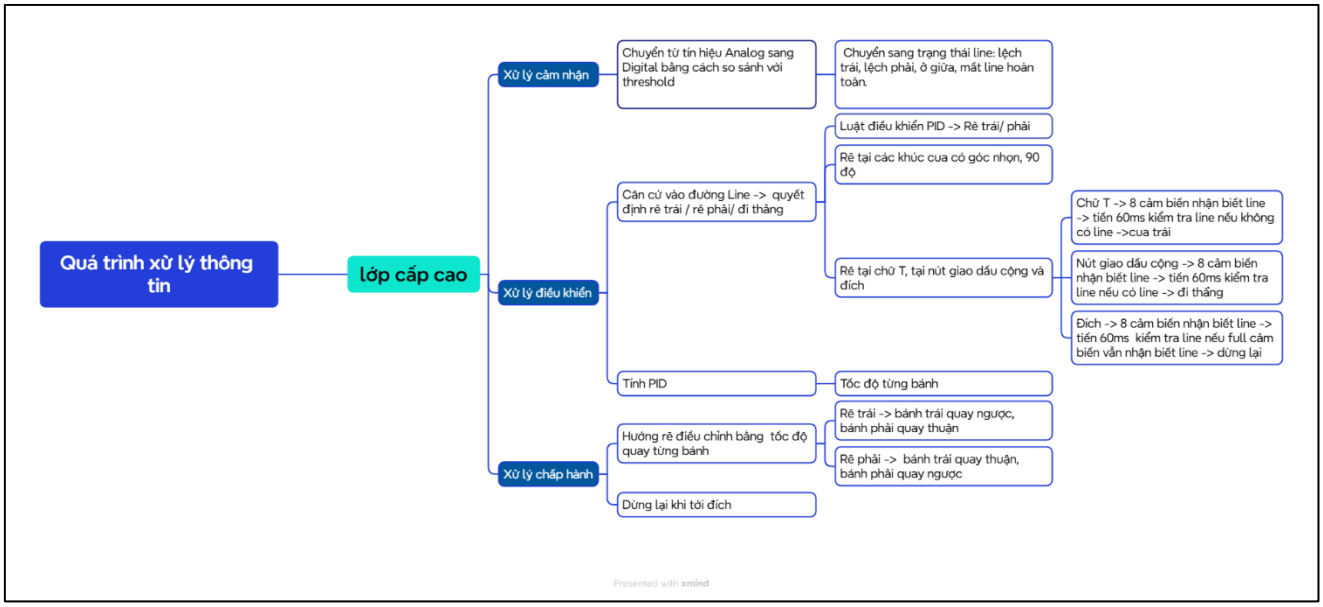
Hình 27: Sơ đồ lớp cấp thấp.

Sơ đồ mô tả logic giao tiếp giữa các thành phần phần cứng thông qua các hàm lập trình:

- **Thiết lập kết nối thành phần phần cứng:** Xác định kết nối vật lý từ cảm biến, vi điều khiển đến các cơ cấu chấp hành.
- **Truyền thông phần cứng:**
    - Sử dụng hàm analogRead() để tiếp nhận dữ liệu liên tục từ 8 mắt cảm biến.

- Sử dụng hàm digitalWrite() và analogWrite() để truyền tín hiệu điều khiển và xung PWM tới Driver động cơ.
- Thiết lập luồng trao đổi dữ liệu từ Driver tới các động cơ thông qua dòng điện DC trực tiếp.

4.2.2. Lớp cấp cao



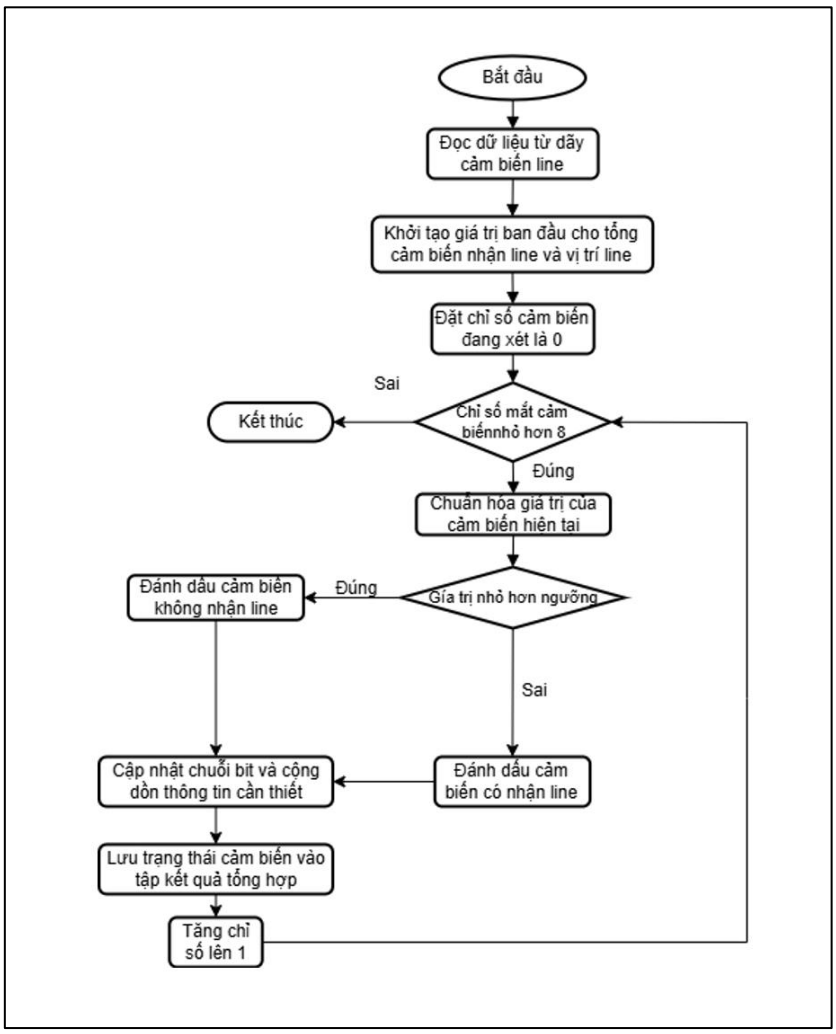
Hình 28: Sơ đồ lớp cấp cao.

Đây là sơ đồ tư duy mô tả logic phần mềm, chia thành 3 giai đoạn xử lý thông tin:

- **Xử lý cảm nhận**:
  - Sử dụng biến threshold để chuyển đổi dữ liệu Analog sang trạng thái nhị phân (0/1).
  - Tính toán vị trí hiện tại thông qua tổng trọng số của các cảm biến để xác định xe đang lệch trái, lệch phải hay ở giữa line.

- **Xử lý điều khiển**:
    - Thuật toán PID: Tính toán giá trị điều chỉnh PID dựa trên sai số hiện tại, sai số tích lũy và tốc độ thay đổi sai số để điều phối tốc độ hai bánh xe mượt mà.
    - Nhận diện cung đường: Xử lý (code) riêng biệt cho góc cua 90 độ, góc nhọn, ngã tư (dấu cộng), ngã ba (chữ T) và vạch đích. Hệ thống sẽ kiểm tra trạng thái của cả 8 cảm biến để đưa ra quyết định rẽ hoặc dừng lại.
- **Xử lý chấp hành**:
    - Điều chỉnh tốc độ: Áp dụng giá trị PID để thay đổi vận tốc bánh trái/phải tương ứng.
    - Hàm điều hướng: Định nghĩa các hàm để điều khiển các bánh xe quay ngược chiều nhau khi cần quay tại chỗ ở các góc gấp.
    - Dừng cơ học: Hàm stop() sẽ ngắt toàn bộ dòng điện tới động cơ khi nhận diện được vùng đích (tất cả cảm biến đều nhận vạch đen).

4.2.3. Sơ đồ khối hoạt động

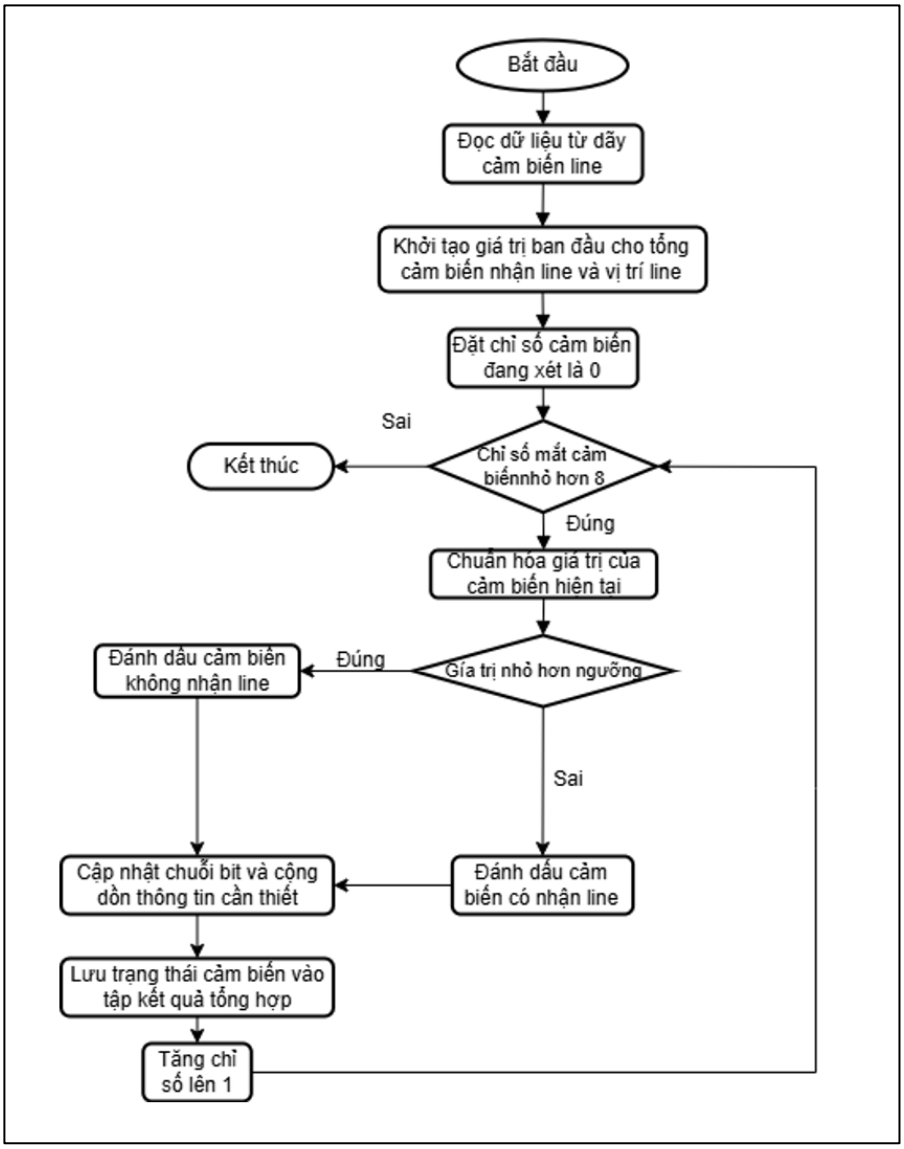


Hình 29: Sơ đồ khối hoạt động của chức năng "Nhận biết đường line"

Sơ đồ này mô tả chi tiết quy trình số hóa dữ liệu trong hàm read_black_line:

- **Khởi tạo chu kỳ**: Reset các biến tích lũy như sumSensor, sumWeight trước khi bắt đầu vòng lặp quét cảm biến.
- **Vòng lặp quét (i < 8)**: Hệ thống thực hiện lặp 8 lần để đọc giá trị Analog của từng mắt cảm biến thứ i.

- **Phân loại dữ liệu**:
    - Nếu giá trị nhỏ hơn ngưỡng (threshold), cảm biến được xác định là nằm ngoài vạch (gán giá trị 0).
    - Nếu giá trị lớn hơn ngưỡng, cảm biến được xác định là đang chạm vạch (gán giá trị 1).
- **Cập nhật hệ thống**: Sau khi quét đủ 8 mắt, dữ liệu được cập nhật vào chuỗi bit và tính tổng để làm đầu vào cho thuật toán PID ở bước tiếp theo.



Hình 30: Sơ đồ khối hoạt động của chức năng "Dò theo đường line"

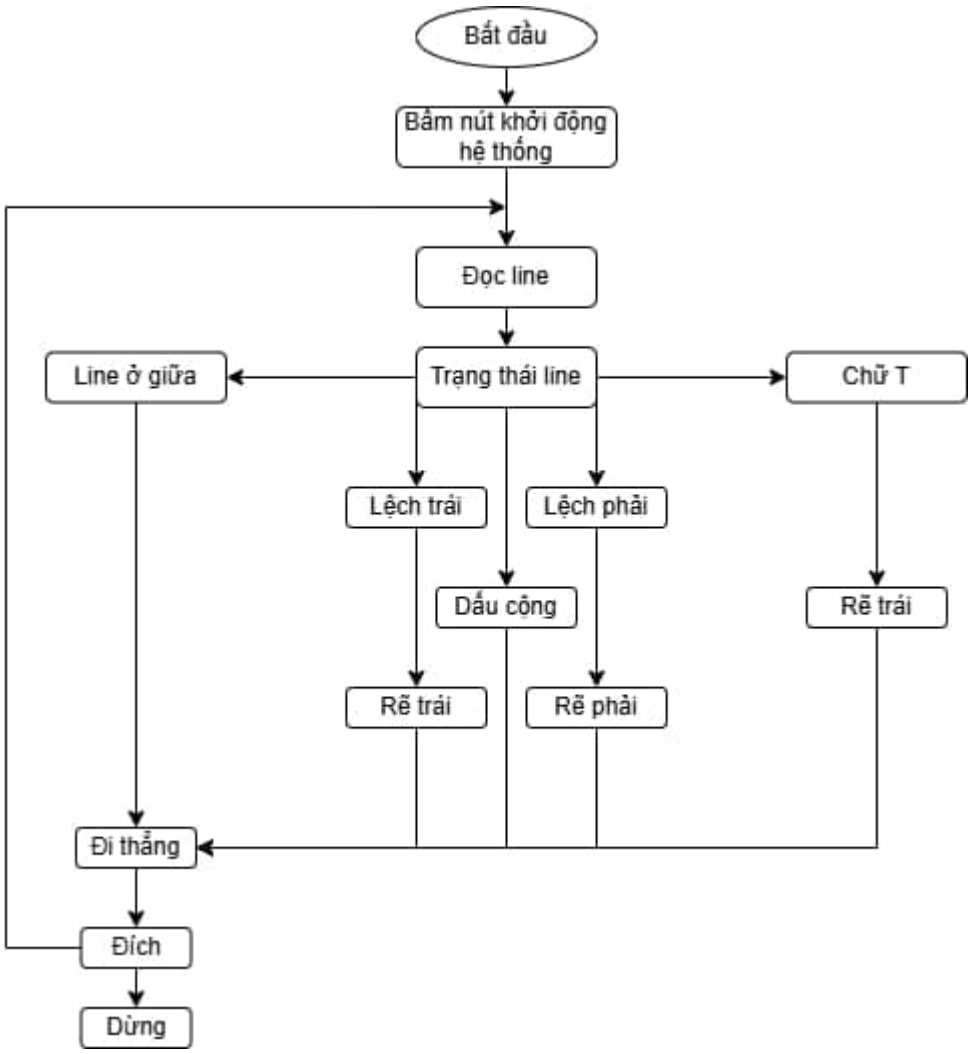
Quy trình thực hiện theo chu kỳ vòng lặp để số hóa dữ liệu:

- **Khởi tạo:** Reset các biến sumSensor, sumWeight.
- **Vòng lặp (i < 8):** Đọc giá trị Analog của cảm biến thứ i.
- **So sánh:** Nếu giá trị nhỏ hơn ngưỡng, gán sensorD = 0 (ngoài vạch); ngược lại gán sensorD = 1 (chạm vạch).
- **Tổng hợp:** Cập nhật chuỗi bit, tính tổng và gán giá trị tương ứng để chuẩn bị cho bước tính toán vị trí.

Mô tả logic vận hành toàn phần của robot:

- **Giai đoạn PID:** Sau khi gọi hàm đọc cảm biến, hệ thống kiểm tra nếu có tín hiệu (sumSensor > 0) sẽ lập tức tính toán qua bộ lọc PID và gọi hàm motor() để điều chỉnh tốc độ hai bánh.
- **Giai đoạn xử lý tình huống:** Chuyển đổi trạng thái cảm biến thành biến bitSensor để đưa vào cấu trúc switch-case:
    - **Trường hợp 255 (gặp ngã rẽ/dấu cộng/đích):** Robot tiến tới 60ms để kiểm tra lại; tùy thuộc vào tín hiệu sau đó mà thực hiện rẽ trái (ngã ba), đi thẳng (dấu cộng) hoặc dừng lại (vùng đích).
    - **Trường hợp 0 (mất line):** chỉ xuất hiện sau **trường hợp 255** robot sẽ tự động gọi lệnh rẽ trái/phải dựa trên hướng di chuyển cuối cùng trước khi mất dấu.
- **Kết thúc:** Hệ thống dừng hoàn toàn khi điều kiện sumSensor = 8 được duy trì sau bước kiểm tra vùng đích.

Sơ đồ khối hoạt động tổng quát:



Hình 31: Sơ đồ khối hoạt động tổng quát.

5. THIẾT KẾ PHẦN MỀM

Thiết kế phần mềm cho xe dò line được lập trình bằng ngôn ngữ C/C++ và xây dựng trên nền tảng phần mềm Arduino IDE cho vi điều khiển Arduino Nano, tập trung vào một vòng lặp điều khiển phản hồi liên tục và chính xác. Các chức năng được module hóa rõ ràng để dễ dàng tinh chỉnh và bảo trì.

5.1. Khai báo cấu hình và biến số toàn cục

Định nghĩa các kết nối phần cứng và thiết lập các tham số điều khiển cốt lõi cho hệ thống robot.

5.1.1. Thiết lập khai báo kết nối phần cứng (Pin config)

Code:

```
#include<SoftwareSerial.h>
SoftwareSerial Bluetooth(10, 11); // RX, TX
#define S1 A7
#define S2 A6
…
#define S8 A0

#define BUTTON1 2
#define BUTTON2 12
#define LED 13
#define PWMA 3
#define AIN1 4
#define AIN2 5
#define STBY 6
#define BIN1 7
#define BIN2 8
#define PWMB 9
```

- **Hoạt động:**
  - Sử dụng thư viện `SoftwareSerial.h` và khai báo `Bluetooth(10, 11)` cho 2 chân 10 và 11 trên HC-05 phục vụ cho việc hiển thị Serial trên điện thoại để debug.
  - Khai báo các chân vật lý trên Arduino Nano sang tên biến gợi nhớ để điều khiển mảng cảm biến và Motor Driver TB6612FNG. Các chân Analog (A0-A7) được gán cho 8 mắt cảm biến để đọc tín hiệu đường line. Các chân Digital (D3-D9) được gán để điều khiển tốc độ (PWM) và hướng quay của hai động cơ trái/phải thông qua Driver.

Và khai báo cho 2 chân sử dụng nút bấm là D2 cho BUTTON1 và D12 cho BUTTON2.

5.1.2. Thiết lập khai báo các biến của khối đọc cảm biến `read_black_line()`

Code:

```
const int sensorNumber = 8;
int sensorPins[sensorNumber] = {S1, S2, S3, S4, S5, S6, S7, S8};
int threshold = 200;
int sumSensor;
int sumWeight;
int senD1, senD2, senD3, senD4, senD5, senD6, senD7, senD8;
int senA1, senA2, senA3, senA4, senA5, senA6, senA7, senA8;
int sensorD[sensorNumber] = {senD1, senD2, senD3, senD4, senD5, senD6, senD7, senD8};
int sensorA[sensorNumber] = {senA1, senA2, senA3, senA4, senA5, senA6, senA7, senA8};
int weightValue[sensorNumber] = {10, 20, 30, 40, 50, 60, 70, 80};
```

- **Hoạt động:**

  - Khai báo `sensorNumber = 8` là tổng số cảm biến trên line sensor
  - `sensorPins[sensorNumber] = {S1, S2, S3, S4, S5, S6, S7, S8}` là mảng gồm các chân của line sensor.
  - Khai báo các biến từ `senA1 -> senA8 và senD1 -> D8` để làm các giá trị trong mảng `sensorA[sensorNumber]` và `sensorD[sensorNumber].`
  - Khai báo mảng `weightValue[sensorNumber]` với các giá trị trong mảng từ 10 -> 80 là trọng số để xác định vị trí cho robot dò line.
  - Khai báo chuỗi `String direction = "straight"` để set default cho switch case sau này.
  - Biến `float cenPos = 45` là vị trí chính giữa của xe, đại lượng này sẽ được đối chiếu với biến `sumWeight` ở hàm `read_black_line()` để xác định sai số `eCurr`. Biến `ePrev` được dùng để tính `dE` là giá trị giữa hiệu hiện tại và sai số của quá khứ dùng để dự đoán lỗi thông qua D (Derivative) của thuật toán PID. `eSum` là biến tính tổng lỗi hiện tại qua các lần. Và cuối cùng là thuật toán PID

được tính bằng `PID = P*eCurr + D*dE + I*eSum`, với P, I, D là các hệ số của thuật toán.

5.1.3. Khai báo các biến của khối điều khiển `PID_control(), turn()`

Code:

```
String direction = "straight";
float currPos;
float cenPos = 45;
float ePrev;
float eCurr;
float eSum;
float PID;
float dE;

int speed = 255;
int turnSpeed = 255;
int move_forward_time = 60;

float P = 17;
float D = 3;
float I = 0;
```

- **Hoạt động:**
  - Khai báo chuỗi `String direction = "straight"` để set default cho switch case sau này.
  - Biến `speed` là tốc độ của xe, `turnSpeed` là tốc độ xoay của xe, `move_forward_time` là thời gian để xe tịnh tiến thêm trong khoảng thời gian 60ms trong các trường hợp xử lý cả 8 cảm biến đều nhận diện màu đen.
  - Biến `float cenPos = 45` là vị trí chính giữa của xe, đại lượng này sẽ được đối chiếu với biến `currPos` ở hàm `PID_control()` để xác định sai số `eCurr`. Biến `ePrev` được dùng để tính `dE` là giá trị giữa hiệu hiện tại và sai số của quá khứ dùng để dự đoán lỗi thông qua D (Derivative) của thuật toán PID.

`eSum` là biến tính tổng lỗi hiện tại qua các lần. Và cuối cùng là thuật toán PID được tính bằng `PID = P*eCurr + D*dE + I*eSum`, với P, I, D là các hệ số của thuật toán.

- Sau khi thử nghiệm và tìm hiểu thêm về thuật toán PID, nhóm thấy rằng hệ số của I không quá hữu dụng trong dự án này vì I hoạt động tốt trong các hệ thống có lỗi trạng thái ổn định. Nhưng trong một robot dò line, robot liên tục quay trái và phải để theo dõi đường đi. Điều này có nghĩa là lỗi không ổn định — nó luôn thay đổi. Nếu chúng ta thêm phần tích phân, nó sẽ tiếp tục tích lũy các điều chỉnh không cần thiết, điều này thực sự gây ra sự dao động và mất ổn định nhiều hơn.

5.2. Hàm cấu hình khởi tạo `setup()`

Hàm này thiết lập các chế độ hoạt động cần thiết cho các chân I/O và các thiết bị ngoại vi, đảm bảo hệ thống sẵn sàng đi vào vòng lặp điều khiển.

```
void setup()
{
  Serial.begin(9600);
  Bluetooth.begin(9600);

  pinMode(BUTTON1, INPUT_PULLUP);
  pinMode(BUTTON2, INPUT_PULLUP);

  pinMode(S1, INPUT);
  pinMode(S2, INPUT);
  pinMode(S3, INPUT);
  pinMode(S4, INPUT);
  pinMode(S5, INPUT);
  pinMode(S6, INPUT);
  pinMode(S7, INPUT);
  pinMode(S8, INPUT);

  pinMode(PWMA, OUTPUT);
  pinMode(AIN1, OUTPUT);
```

```
  pinMode(AIN2, OUTPUT);
  pinMode(STBY, OUTPUT);
  pinMode(BIN2, OUTPUT);
  pinMode(BIN1, OUTPUT);
  pinMode(PWMB, OUTPUT);

  pinMode(LED, OUTPUT);

  digitalWrite(STBY, LOW);
  digitalWrite(BUTTON1, HIGH);
  digitalWrite(BUTTON2, HIGH);
}
```

5.2.1. Khởi tạo giao tiếp và cổng Serial

Code:

```
Serial.begin(9600);
Bluetooth.begin(9600);
```

- **Hoạt động:**
  - `Serial.begin()` được set ở mức baudrate 9600 để có thể in ra màn hình Serial phục vụ việc theo dõi thông số cảm biến, xung của motor để thực hiện debug.
  - `Bluetooth.begin` có vai trò tương tự nhưng có điểm khác biệt là thay vì truyền lên màn hình Serial trong IDE thì nó có thể truyền lên Serial trên app điện thoại bằng bluetooth thông qua module HC-05.

5.2.2. Cấu hình chế độ chân cho cảm biến và nút bấm

Code:

```
pinMode(BUTTON1, INPUT_PULLUP);
pinMode(BUTTON2, INPUT_PULLUP);
pinMode(S1, INPUT);
...
```

```
pinMode(S8, INPUT);
```

- **Hoạt động:**
  - Các nút bấm được thiết lập `INPUT_PULLUP` để sử dụng điện trở kéo lên nội bộ của Arduino, giúp nhận diện trạng thái nhấn nút một cách ổn định mà không cần thêm linh kiện bên ngoài.
  - Các chân S1 đến S8 được đặt ở chế độ `INPUT` nhận tín hiệu để đọc tín hiệu từ các mắt thu hồng ngoại.

5.2.3. Cấu hình chân điều khiển động cơ và Driver

Code:

```
pinMode(PWMA, OUTPUT);
pinMode(AIN1, OUTPUT);
pinMode(AIN2, OUTPUT);
... tương tự cho kênh B
pinMode(STBY, OUTPUT);
```

- **Hoạt động:**
  - `PWMA, AIN1, AIN2` và `PWMB, BIN1, BIN2` dùng để gửi tín hiệu xung cho driver TB6612FNG, từ đó driver xuất ra tín hiệu xung để điều khiển động cơ cho 2 motor qua các chân `AO1, AO2 và BO1, BO2` trên driver.
  - `STBY` dùng để ra lệnh bật tắt động cơ.

5.2.4. Kích hoạt trạng thái ban đầu của hệ thống

Code:

```
pinMode(LED, OUTPUT);
digitalWrite(STBY, LOW);
digitalWrite(BUTTON1, HIGH);
digitalWrite(BUTTON2, HIGH);
```

- **Hoạt động:**
  - Chân `STBY` được kéo về mức `LOW` để tạm thời tắt driver động cơ, ngăn chặn việc xe tự di chuyển ngoài ý muốn trước khi có lệnh bắt đầu.
  - Các chân nút bấm được duy trì ở mức `HIGH` để chuẩn bị sẵn sàng nhận tín hiệu kích hoạt từ người dùng.
  - Chân `LED` dùng để làm tín hiệu debug cho các trường hợp xử lý đặc biệt là rẽ tại các góc 90 độ, góc nhọn, chữ T, dấu cộng và điểm đích.

5.3. Vòng lặp chính `loop()`

Đây là vòng lặp vô tận, nơi chương trình điều khiển phản hồi được thực thi liên tục để duy trì hoạt động của robot.

Code:

```
void loop()
{
if(digitalRead(BUTTON1)==0)
  {
    delay(1000);
    digitalWrite(STBY,HIGH);
    while(1)
    {
      read_black_line();
      PID_control(P, I, D);
      delay(5);
    }
  }
  if(digitalRead(BUTTON2)==0)
  {
    while(1)
    {
      debug();
    }
  }
}
```

5.3.1. Chế độ vận hành (BUTTON1)

Code:

```
if(digitalRead(BUTTON1)==0)
{
  delay(1000);
  digitalWrite(STBY,HIGH);
  while(1)
  {
    read_black_line();
    PID_control(P, I, D);
    delay(5);
  }
}
```

- **Hoạt động:**

  - Khi nhấn `BUTTON1`, hệ thống tạm dừng 1000ms (1s) để người dùng ổn định vị trí robot trên vạch xuất phát.

  - Lệnh `digitalWrite(STBY, HIGH)` bật driver TB6612FNG, cho phép motor bắt đầu hoạt động.

  - Vòng lặp vô hạn `while(1)` sẽ bắt đầu sau khi bấm `BUTTON1`, trong đó hàm `read_black_line()` và sau đó `PID_control(P, I, D)` được gọi liên tục. Điều này có thể nghĩa là việc điều khiển PID sẽ liên tục chạy mà không có kết thúc, điều khiển liên tục cho hệ thống. Hệ thống sẽ đọc dữ liệu cảm biến bằng hàm `read_black_line()` và cập nhật dữ liệu đó cho hàm `PID_control(P, I, D)` tính toán để chấp hành cho động cơ.

    - Nhưng hàm `loop()` chạy rất nhanh, việc chạy liên tục như vậy sẽ tiêu tốn quá nhiều tài nguyên vi điều khiển. Dù không gây ra lỗi nghiêm trọng, việc chạy liên tục có thể làm cho vi điều khiển

quá tải hoặc khó khăn trong việc đồng bộ các tác vụ khác. Vì vậy nhóm đã thêm một hàm nữa dưới hàm `PID_control()` một hàm `delay(5),` hàm này có tác dụng tạo ra một khoảng trễ, một khoảng nghỉ ngắn trong vòng lặp `loop()`, giúp cho vi điều khiển tránh trường hợp bị quá tải nhưng vẫn đáp ứng đủ nhanh để thực hiện đầy đủ các chức năng của xe.

- Việc không có `delay()` có thể khiến các tín hiệu PWM (điều khiển motor) hoặc cảm biến không kịp cập nhật kịp thời, gây ra hiện tượng tín hiệu chồng chéo hoặc không đồng bộ. Với `delay(5)`, bạn có một khoảng thời gian nhỏ để đảm bảo hệ thống không bị quá tải và có thể xử lý tín hiệu một cách chính xác.
- Một phần khác của việc sử dụng `delay(5)` là giới hạn tốc độ tính toán trong vòng lặp PID. Nếu vòng lặp chạy quá nhanh, PID có thể điều chỉnh tốc độ động cơ quá nhanh, dẫn đến hiệu suất không tối ưu. `delay(5)` giúp làm chậm quá trình một chút để PID có thể hoạt động ổn định hơn.

5.3.2. Chế độ kiểm tra dữ liệu cảm biến (BUTTON2)

Code:

```
if(digitalRead(BUTTON2)==0)
  {
    while(1)
    {
      debug();
    }
  }
```

- **Hoạt động:**
  - Tương tự với `BUTTON1`, hàm `debug()` sẽ được gọi liên tục. Hàm `debug()` chính là hàm `read_black_line()` nhưng thêm các lệnh `Serial.print` để thực hiện debug. Việc có thêm 1 nút bấm nữa để

dành riêng cho hàm `debug()` này là để tránh chạy Serial trong lúc thực thi hàm chuyển động `PID_control()`.

- Vì lượng Serial in ra có thể gây quá tả cho quá trình xử lý, khiến cho xe chạy không như mong muốn. Có thêm 1 nút bấm riêng cũng khiến cho quá trình vận hành trở nên tiện lợi hơn, đỡ phải xoá đi `Serial.print` mỗi khi chạy xe.

5.4. Hàm xử lý tín hiệu cảm biến `read_black_line()`

Hàm này chuyển đổi tín hiệu Analog vật lý từ các mắt cảm biến hồng ngoại thành dữ liệu vị trí để robot nhận diện đường đi.

```
void read_black_line()
{
  sumSensor = 0;
  sumWeight = 0;

  for (int i = 0; i < sensorNumber; i++)
  {
    sensorA[i] = analogRead(sensorPins[i]);

    if(sensorA[i] < 200)
    {
      sensorA[i] = 0;
    }
    else if(sensorA[i] > 300)
    {
      sensorA[i] = 1023;
    }
    sensorD[i] = (sensorA[i] < threshold) ? 0 : 1;
    sumSensor += sensorD[i];
    sumWeight += sensorD[i] * weightValue[i];

    if (i == 0) senD1 = sensorD[i];
    if (i == 1) senD2 = sensorD[i];
    if (i == 2) senD3 = sensorD[i];
```

```
    if (i == 3) senD4 = sensorD[i];
    if (i == 4) senD5 = sensorD[i];
    if (i == 5) senD6 = sensorD[i];
    if (i == 6) senD7 = sensorD[i];
    if (i == 7) senD8 = sensorD[i];
  }
}
```

Vòng lặp `for` với `i = 0` và `sensorNumber = 8` sẽ quét qua từng cảm biến trên line sensor từ S1 -> S8.

5.4.1. Khởi tạo và làm sạch dữ liệu chu kỳ mới

Code:

```
sumSensor = 0;
sumWeight = 0;
```

- **Hoạt động:** Đưa các biến tích lũy về giá trị mặc định trước khi bắt đầu một lượt quét cảm biến mới. Việc thiết lập lại `sumSensor` và `sumWeight` về 0 giúp ngăn chặn hiện tượng dữ liệu bị cộng dồn sai lệch giữa các chu kỳ đọc, đảm bảo tính chính xác cho thuật toán tính vị trí.

5.4.2. Đọc tín hiệu tương tự và chuyển đổi nhị phân

Code:

```
sensorA[i] = analogRead(sensorPins[i]);

if(sensorA[i] < 200)
{
  sensorA[i] = 0;
}
else if(sensorA[i] > 300)
{
  sensorA[i] = 1023;
}
sensorD[i] = (sensorA[i] < threshold) ? 0 : 1;
```

- **Hoạt động:**

  - Hàm `analogRead()` thu thập giá trị Analog phản xạ từ chân S1 đến S8 lưu vào mảng `sensorA`.

```
22:34:32.282 -> SensorA: 134 134 135 138 136 137 135 139
22:34:32.315 -> SensorA: 135 136 134 139 137 136 139 139
22:34:32.416 -> SensorA: 132 132 137 137 138 135 138 139
22:34:32.416 -> SensorA: 133 133 137 137 135 134 135 135
22:34:32.448 -> SensorA: 134 134 137 135 140 137 134 139
```

Hình 32: Analog 8 cảm biến khi ở nền trắng

```
22:30:53.353 -> SensorA: 514 507 544 539 532 535 532 525
22:30:53.386 -> SensorA: 515 506 544 540 532 534 533 523
22:30:53.420 -> SensorA: 513 507 543 540 533 535 533 524
22:30:53.487 -> SensorA: 514 507 546 540 529 535 534 523
22:30:53.553 -> SensorA: 511 507 544 539 531 533 532 522
```

Hình 33: Analog 8 cảm biến khi ở line đen

  - Sau khi đọc được dữ liệu Analog raw từ line sensor, sử dụng tiếp 2 lệnh `if` và `else if` để scale giá trị Analog về khoảng [0, 1023]. Nếu bất kỳ cảm biến nào trên line sensor trả về kết quả bé hơn 200 sẽ scale về 0, nếu lớn hơn 300 sẽ trả về 1023. Việc scale lại khoảng như vậy sẽ giúp xét ngưỡng threshold hiệu quả hơn. Việc lấy giá trị ở 2 đầu ngưỡng mới có thể lấy giá trị khác, chỉ cần không trùng với giá trị xét và trùng với những giá trị trong khoảng của Analog raw.

```
22:55:16.218 -> SensorA: 0 0 0 0 0 0 0 0
22:55:16.252 -> SensorA: 0 0 0 0 0 0 0 0
22:55:16.285 -> SensorA: 0 0 0 0 0 0 0 0
22:55:16.352 -> SensorA: 0 0 0 0 0 0 0 0
22:55:16.352 -> SensorA: 0 0 0 0 0 0 0 0
```

Hình 34: Analog 8 cảm biến khi ở line trắng (đã scale)

```
22:57:49.571 -> SensorA: 1023 1023 1023 1023 1023 1023 1023 1023
22:57:49.638 -> SensorA: 1023 1023 1023 1023 1023 1023 1023 1023
22:57:49.672 -> SensorA: 1023 1023 1023 1023 1023 1023 1023 1023
22:57:49.738 -> SensorA: 1023 1023 1023 1023 1023 1023 1023 1023
22:57:49.805 -> SensorA: 1023 1023 1023 1023 1023 1023 1023 1023
```

Hình 35: Analog 8 cảm biến khi ở line trắng (đã scale)

- Toán tử điều kiện so sánh giá trị đọc được đã scale với `threshold` để chuyển đổi qua Digital: Nếu nhỏ hơn ngưỡng (nền trắng), cảm biến nhận giá trị 0; ngược lại nhận giá trị 1 (line đen). Ngưỡng hiện `threshold` tại nhóm đang chọn là 200. Việc chuyển đổi qua digital để phục vụ cho việc xử lý các trường hợp chuyển động của xe.

```
23:07:23.661 -> SensorD: 0 0 0 0 0 0 0 0
23:07:23.726 -> SensorD: 0 0 0 0 0 0 0 0
23:07:23.726 -> SensorD: 0 0 0 0 0 0 0 0
23:07:23.762 -> SensorD: 0 0 0 0 0 0 0 0
23:07:23.762 -> SensorD: 0 0 0 0 0 0 0 0
```

Hình 36: Digital 8 cảm biến khi ở line trắng

```
23:07:35.347 -> SensorD: 1 1 1 1 1 1 1 1
23:07:35.380 -> SensorD: 1 1 1 1 1 1 1 1
23:07:35.414 -> SensorD: 1 1 1 1 1 1 1 1
23:07:35.414 -> SensorD: 1 1 1 1 1 1 1 1
23:07:35.447 -> SensorD: 1 1 1 1 1 1 1 1
```

Hình 37: Digital 8 cảm biến khi ở line đen

5.4.3. Tính toán tổng tích lũy trọng số

Code:

```
sumSensor += sensorD[i];
sumWeight += sensorD[i] * weightValue[i];
```

- **Hoạt động:**
  - Tích lũy các giá trị cần thiết để xác định vị trí tương đối của robot so với đường line.

- `sumSensor:` Đếm tổng số lượng mắt cảm biến đang nhận diện được vạch đen.

```
23:24:31.840 -> sumSensor và SensorD: 0  0 0 0 0 0 0 0 0
23:24:31.874 -> sumSensor và SensorD: 0  0 0 0 0 0 0 0 0
23:24:31.940 -> sumSensor và SensorD: 0  0 0 0 0 0 0 0 0
23:24:31.974 -> sumSensor và SensorD: 0  0 0 0 0 0 0 0 0
23:24:32.007 -> sumSensor và SensorD: 0  0 0 0 0 0 0 0 0
```

Hình 38: Tổng số cảm biến khi không có cảm biến nào phát hiện line

```
23:23:32.742 -> sumSensor và SensorD: 8  1 1 1 1 1 1 1 1
23:23:32.809 -> sumSensor và SensorD: 8  1 1 1 1 1 1 1 1
23:23:32.843 -> sumSensor và SensorD: 8  1 1 1 1 1 1 1 1
23:23:32.876 -> sumSensor và SensorD: 8  1 1 1 1 1 1 1 1
23:23:32.943 -> sumSensor và SensorD: 8  1 1 1 1 1 1 1 1
23:23:32.976 -> sumSensor và SensorD: 8  1 1 1 1 1 1 1 1
```

Hình 39: Tổng số cảm biến khi 8 có cảm biến nào phát hiện line

```
23:32:48.650 -> sumWeight và SensorD: 90  0 0 0 1 1 0 0 0
23:32:48.717 -> sumWeight và SensorD: 90  0 0 0 1 1 0 0 0
23:32:48.751 -> sumWeight và SensorD: 90  0 0 0 1 1 0 0 0
23:32:48.784 -> sumWeight và SensorD: 90  0 0 0 1 1 0 0 0
23:32:48.850 -> sumWeight và SensorD: 90  0 0 0 1 1 0 0 0
```

Hình 40: Khi xe ở vị trí trung tâm

- `sumWeight:` Cộng dồn trọng số của những mắt đang nằm trên vạch đen theo mảng `weightValue` đã khai báo với các giá trị trong mảng từ 10 -> 80.

```
23:35:06.477 -> sumWeight và SensorD: 360  1 1 1 1 1 1 1 1
23:35:06.544 -> sumWeight và SensorD: 360  1 1 1 1 1 1 1 1
23:35:06.578 -> sumWeight và SensorD: 360  1 1 1 1 1 1 1 1
23:35:06.611 -> sumWeight và SensorD: 360  1 1 1 1 1 1 1 1
23:35:06.678 -> sumWeight và SensorD: 360  1 1 1 1 1 1 1 1
```

```
23:35:23.573 -> sumWeight và SensorD: 0  0 0 0 0 0 0 0 0
23:35:23.606 -> sumWeight và SensorD: 0  0 0 0 0 0 0 0 0
23:35:23.672 -> sumWeight và SensorD: 0  0 0 0 0 0 0 0 0
23:35:23.706 -> sumWeight và SensorD: 0  0 0 0 0 0 0 0 0
23:35:23.739 -> sumWeight và SensorD: 0  0 0 0 0 0 0 0 0
```

Hình 42: Khi 8 cảm biến ở nền trắng

5.4.4. Lưu các giá trị trong mảng `sensorD[i]` vào các giá trị tương ứng

Code:

```
if (i == 0) senD1 = sensorD[i];
if (i == 1) senD2 = sensorD[i];
...
if (i == 7) senD8 = sensorD[i];
```

- **Hoạt động:** Gán dữ liệu từ mảng vào các biến độc lập để sử dụng vào việc xử lý chuyển động trong hàm `PID_control()`.

5.5. Hàm điều khiển động cơ `motor()`

Hàm này là dùng nhận tín hiệu từ driver TB6612FNG, để nhận xung chấp hành cho motor hoạt động.

Code:

```
void motor(int LPWM, int RPWM)
{
  LPWM = constrain(LPWM, -255, 255);
  RPWM = constrain(RPWM, -255, 255);

  if (LPWM >= 0)
  {
    digitalWrite(AIN1, LOW);
    digitalWrite(AIN2, HIGH);
  }
  else
  {
    digitalWrite(AIN1, HIGH);
    digitalWrite(AIN2, LOW);
  }
```

```
  if (RPWM >= 0)
  {
    digitalWrite(BIN1, HIGH);
    digitalWrite(BIN2, LOW);
  }
  else
  {
    digitalWrite(BIN1, LOW);
    digitalWrite(BIN2, HIGH);
  }

  analogWrite(PWMA, abs(LPWM));
  analogWrite(PWMB, abs(RPWM));
}
```

5.5.1. Giới hạn và chuẩn hóa tín hiệu đầu vào

Code:

```
LPWM = constrain(LPWM, -255, 255);
RPWM = constrain(RPWM, -255, 255);
```

- **Hoạt động:** Đảm bảo các giá trị tốc độ tính toán luôn nằm trong phạm vi xử lý an toàn của phần cứng. Hàm `constrain` ép các giá trị `LPWM` (bánh trái) và `RPWM` (bánh phải) không vượt quá dải từ -255 đến 255. Điều này ngăn chặn hiện tượng tràn bộ nhớ hoặc gửi các tín hiệu không hợp lệ làm lỗi trình điều khiển động cơ.

5.5.2. Điều khiển hướng quay động cơ bên trái

Code:

```
if (LPWM >= 0)
{
    digitalWrite(AIN1, LOW);
    digitalWrite(AIN2, HIGH);
}
```

```
else
{
    digitalWrite(AIN1, HIGH);
    digitalWrite(AIN2, LOW);
}
```

- **Hoạt động:**

    - Thiết lập trạng thái logic cho các chân điều khiển hướng quay của động cơ trái. Nếu tốc độ lớn hơn hoặc bằng 0, chân `AIN1/AIN2` được đặt là `LOW/HIGH` để bánh xe quay tiến. Ngược lại, nếu giá trị âm, các chân này sẽ đảo ngược trạng thái để bánh xe quay lùi.

5.5.3. Điều khiển hướng quay động cơ bên phải

Code:

```
if (RPWM >= 0)
{
    digitalWrite(BIN1, HIGH);
    digitalWrite(BIN2, LOW);
}
else
{
    digitalWrite(BIN1, LOW);
    digitalWrite(BIN2, HIGH);
}
```

- **Hoạt động:**

    - Thiết lập trạng thái logic cho các chân điều khiển hướng quay của động cơ trái. Nếu tốc độ lớn hơn hoặc bằng 0, chân `BIN1/BIN2` được đặt là `HIGH/LOW` để bánh xe quay tiến. Ngược lại, nếu giá trị âm, các chân này sẽ đảo ngược trạng thái để bánh xe quay lùi.

    - Lý do `AIN1/AIN2` được đặt là `LOW/HIGH` mà `BIN1/BIN2` lại đặt là `HIGH/LOW` là do hai motor được đặt ở 2 vị trí đối và ngược chiều

nhau, nên 1 bánh sẽ phải xoay thuận và một bánh phải xoay nghịch thì xe mới có thể đi thẳng được.

5.5.4. Xuất tín hiệu tốc độ thực tế (PWM)

Code:

```
analogWrite(PWMA, abs(LPWM));
analogWrite(PWMB, abs(RPWM));
```

- **Hoạt động:** Gửi tín hiệu PWM tới chân điều khiển động cơ, điều chỉnh tốc độ động cơ. Hàm abs() để chuyển các giá trị LPWM và RPWM thành giá trị dương để điều khiển tốc độ, bất kể giá trị ban đầu là âm hay dương.

5.6. Hàm dừng động cơ `stop()`

```
void stop()
{
  motor(0,0);
  digitalWrite(LED,HIGH);
}
```

- **Hoạt động:** Hàm có tác dụng dừng động cơ khi được gọi ở trường hợp khi xe về đích và bật đèn LED báo trạng thái.

5.7. Hàm dừng động cơ `move_forward(unsigned long moveTime)`

```
void move_forward(int moveTime)
{
  int startTime = millis();
  while (millis() - startTime < moveTime)
  {
    motor(speed, speed);
  }
}
```

- **Hoạt động:**
    - Khai báo biến `moveTime` là thời gian xe sẽ tịnh tiến lên 1 khoảng trong thời gian `moveTime` hay chính là `move_forward_time = 60` đã khai báo ngay từ đầu để xử lý tại các trường hợp đặc biệt là chữ T, dấu cộng và điểm đích.
    - Khai báo `startTime = millis()` để lưu thời gian bắt đầu của hàm từ khi hàm bắt đầu được thực thi.
    - Vòng lặp `while` với điều kiện: `millis() - startTime < moveTime`, `millis()` là hàm của Arduino trả về số mili-giây (ms) đã trôi qua kể từ lúc board bắt đầu chạy chương trình. millis() trừ đi thời gian `startTime` để kiểm tra xem thời gian của xe ở trên line đen khi cả 8 cảm biến cùng phát hiện ra line đen là bao lâu. Nếu điều kiện vẫn đúng thì sẽ thực thi hàm `motor(speed, speed)`, nếu sai thì `while` sẽ `break` và thực thi các lệnh phía sau.

5.8. Hàm rẽ trái và rẽ phải `turnRight(), turnLeft()`

Code:

```
void turnRight()
{
  while(sensorD[4] != 1)
  {
    motor(turnSpeed, -turnSpeed);
    digitalWrite(LED,HIGH);
    read_black_line();
    direction = "straight";
  }
  digitalWrite(LED,LOW);
}
void turnLeft()
{
  while(sensorD[4] != 1)
  {
    motor(-turnSpeed, turnSpeed);
```

```
    digitalWrite(LED,HIGH);
    read_black_line();
    direction = "straight";
  }
  digitalWrite(LED,LOW);
}
```

- Hoạt động:
  - 2 hàm này sẽ phục vụ cho phần chuyển động ở hàm `PID_control()` sẽ được nhắc tới ở mục tới.
  - Chức năng của nó sẽ là xoay ngược chiều kim đồng hồ với `turnLeft()` và theo chiều kim đồng hồ với `turnRight()`. Nhưng để nếu chỉ đơn thuần là xoay tròn như vậy thì nó vẫn sẽ không thể rẽ được mà chỉ xoay tròn ở một chỗ.
  - Do đó nhóm cần thêm điều kiện cho chức năng này và nhóm dùng vòng lặp `while(sensorD[4] != 1)`. Vòng lặp cùng điều kiện này dùng để định hình lại hướng đi của xe khi xoay. Vòng lặp sẽ chạy tới khi cảm biến thứ 5 không còn là giá trị 1 nữa, nghĩa là giá trị sẽ bằng 0 và biểu hiện của không phát hiện ra line ở cảm biến thứ 5 đó.
  - Khi điều kiện trong vòng lặp vẫn đúng, với `turnRight()` nó sẽ thực thi hàm `motor(turnSpeed, -turnSpeed)`,với `turnSpeed = 255` đã khai báo ở đầu chương trình, sẽ làm cho motor xoay theo chiều kim đồng hồ và hướng xe sang bên phải, tương tự với turnLeft là `motor(-turnSpeed, turnSpeed)` xoay ngược chiều kim đồng hồ và hướng xe sang bên trái, lúc này đèn LED sẽ bật để thông báo rẽ.
  - Sau đó sử dụng hàm `read_black_line()` để tiếp tục cập nhật thay đổi của cảm biến. Sau khi hàm `read_black_line()` được thực thi mà nó phát hiện ra `sensorD[4] != 1` không còn đúng nữa, xe sẽ được set về `direction = “straight”` để tiếp tục đi thẳng được xử lý bởi PID và nó sẽ `break` vòng lập, tắt đèn LED báo hiệu quá trình rẽ đã hoàn tất.

5.9. Hàm điều khiển và cân bằng động cơ `PID_control(float Kp, float Ki, float Kd)`

Code:

5.9.1. Thuật toán PID và chấp hành cho `motor()`

```
PID_control(float Kp, float Ki, float Kd)
{
 if(sumSensor > 0)
 {
    currPos = sumWeight / sumSensor;
    eCurr = cenPos - currPos;
    dE = eCurr - ePrev;
    eSum += eCurr;

    PID = Kp * eCurr + Kd * dE + Ki * eSum;

    int leftSpeed  = speed - PID;
    int rightSpeed = speed + PID;

    ePrev = eCurr;
    motor(leftSpeed, rightSpeed);
 }
```

- **Hoạt động:**
  - Trong hàm loop(), sau khi đã thực thi hàm read_black_line() xong, tiếp sau đó sẽ tới hàm PID_control(), việc đọc cảm biến có thể kiểm tra được có ít nhất một cảm biến trên đường line qua điều kiện if(sumSensor > 0) để có thể bắt đầu thực thi hàm PID_control().
  - `currPos:` Tính toán vị trí hiện tại của vạch đen dựa trên dữ liệu từ hàm đọc cảm biến. Như theo công thức ta có `sumWeight = 90` và `sumSensor` như dưới hình là 00011000 có nghĩa là bằng 2 nên suy ra `currPos = 45`

```
02:02:05.705 -> sumWeight và eCurr và SensorD: 90  45.00  0 0 0 1 1 0 0 0
02:02:05.805 -> sumWeight và eCurr và SensorD: 90  45.00  0 0 0 1 1 0 0 0
02:02:05.839 -> sumWeight và eCurr và SensorD: 90  45.00  0 0 0 1 1 0 0 0
02:02:05.906 -> sumWeight và eCurr và SensorD: 90  45.00  0 0 0 1 1 0 0 0
02:02:06.006 -> sumWeight và eCurr và SensorD: 90  45.00  0 0 0 1 1 0 0 0
```

Hình 43: Vị trí của xe khi ở chính giữa line

- `eCurr`: Sai số hiện tại được xác định bằng cách lấy điểm đích lý tưởng (`cenPos = 45`) trừ đi vị trí thực tế (`currPos`). Sai số có thể lớn hơn trong hình, phụ thuộc vào độ lệch hiện tại nhiều hay ít của xe.

```
02:12:22.522 -> currPos và eCurr và SensorD: 45.00  0.00  0 0 0 1 1 0 0 0
02:12:22.589 -> currPos và eCurr và SensorD: 45.00  0.00  0 0 0 1 1 0 0 0
02:12:22.655 -> currPos và eCurr và SensorD: 45.00  0.00  0 0 0 1 1 0 0 0
02:12:22.723 -> currPos và eCurr và SensorD: 45.00  0.00  0 0 0 1 1 0 0 0
02:12:22.789 -> currPos và eCurr và SensorD: 45.00  0.00  0 0 0 1 1 0 0 0
```

Hình 44: Sai số của xe khi ở chính giữa line

```
02:14:45.124 -> currPos và eCurr và SensorD: 50.00  -5.00  0 0 0 0 1 0 0 0
02:14:45.191 -> currPos và eCurr và SensorD: 50.00  -5.00  0 0 0 0 1 0 0 0
02:14:45.258 -> currPos và eCurr và SensorD: 50.00  -5.00  0 0 0 0 1 0 0 0
02:14:45.324 -> currPos và eCurr và SensorD: 50.00  -5.00  0 0 0 0 1 0 0 0
02:14:45.391 -> currPos và eCurr và SensorD: 50.00  -5.00  0 0 0 0 1 0 0 0
```

Hình 45: Sai số của xe khi cảm biến lệch sang bên phải 1 cảm biến

```
currPos và eCurr và SensorD: 40.00  5.00  0 0 0 1 0 0 0 0
currPos và eCurr và SensorD: 40.00  5.00  0 0 0 1 0 0 0 0
currPos và eCurr và SensorD: 40.00  5.00  0 0 0 1 0 0 0 0
currPos và eCurr và SensorD: 40.00  5.00  0 0 0 1 0 0 0 0
currPos và eCurr và SensorD: 40.00  5.00  0 0 0 1 0 0 0 0
```

Hình 46: Sai số của xe khi cảm biến lệch sang bên trái 1 cảm biến

- `eSum`: Như nhóm cũng đã giải thích ở phần 5.1.3, nhóm sẽ không sử dụng đại lượng I (tích phân) trong dự án này, vì vậy eSum cũng không sử dụng.

- `dE`: Tính toán biến thiên sai số, lấy sai số hiện tại (`eCurr`) để trừ đi sai số trước đó (`ePrev`), từ đó dự đoán trước được sai số và và chọn 1 hệ số D phù hợp để xe hoạt động mượt mà.

- Phương trình `PID = Kp*eCurr + Kd*dE + Ki*eSum (Ki = 0)`. Các hệ số Kp và Kd để quyết định độ mạnh – yếu của phần xử lý của mình.

  - **Thành phần Tỉ lệ (Kp):** Tạo phản ứng trực tiếp theo độ lệch. Sai số càng lớn, lực bẻ lái càng mạnh để xe phản ứng tức thời. Nhưng nếu để Kp quá thấp, phản ứng sẽ rất yếu, không thể tìm về set point một cách nhanh chóng được, nếu Kp quá cao thời gian phản ứng sẽ rất nhanh, tìm về set point nhanh chóng nhưng với đó sẽ dễ bị vọt lố (overshoot).

  - **Thành phần Dẫn xuất (Kd):** Dự báo xu hướng thay đổi của sai số, đóng vai trò như bộ giảm chấn để triệt tiêu quán tính, giúp xe không bị lắc lư (overshoot) khi đi thẳng. Nếu Kd quá thấp, không đủ để triệt tiêu lắc lư do vọt lố gây ra, nếu Kd quá cao thì làm cho xe liên tục điều chỉnh và sẽ giảm hiệu suất của xe.

- Khai báo 2 biến `leftSpeed` và `rightSpeed` để bù trừ sai lệch cho từ bánh thông qua PID đã tính. `leftSpeed = speed - PID` và `rightSpeed = speed + PID`. Lý do không phải cộng trừ ngược lại đó là do khi xe lệch sang trái thì sai số sẽ âm, lệch sang phải sai số sẽ dương. Ví dụ: Khi xe đang lệch sang bên trái sai số sẽ âm, nên PID tính ra cũng sẽ âm, mà theo công thức của `leftSpeed` và `rightSpeed` với `speed` đã khai báo từ đầu tiên là 255, vì vậy bánh trái sẽ tăng tốc và bánh phải giảm tốc khiến cho xe được đẩy lại về bên phải bù lại khoảng lệch trái từ ban đầu.

```
02:54:45.674 ->  eCurr || LRspeed || PID || SensorD: -15.00  480  30  -225.000 0 0 0 0 1 0 0
02:54:45.775 ->  eCurr || LRspeed || PID || SensorD: -15.00  480  30  -225.000 0 0 0 0 1 0 0
02:54:45.841 ->  eCurr || LRspeed || PID || SensorD: -15.00  480  30  -225.000 0 0 0 0 1 0 0
02:54:45.975 ->  eCurr || LRspeed || PID || SensorD: -15.00  480  30  -225.000 0 0 0 0 1 0 0
02:54:46.008 ->  eCurr || LRspeed || PID || SensorD: -15.00  480  30  -225.000 0 0 0 0 1 0 0
```

Hình 47: PID và tốc độ từng bánh khi lệnh sang bên trái

- Cuối cùng là hệ số sử dụng cho P, I, D. Như ở ngay đầu chương trình đã khai báo, ba hệ số mà nhóm sau khi tuning là `P = 17, I = 0, D = 3.`

Cách để nhóm chọn ra được các hệ số này là dựa trên phương pháp **Empirical PID gain tuning**. Đây là một phương pháp điều chỉnh các tham số của bộ điều khiển PID dựa trên thử nghiệm và quan sát thực tế, thay vì sử dụng các công thức toán học phức tạp. Nhóm đã thực hiện thử nghiệm nhiều lần và sử dụng mô phỏng Python để đưa ra được các hệ số phù hợp cho xe theo quy trình sau:

- Đầu tiên là thử nghiệm với hệ số P, nhóm sẽ bắt đầu từ P = 1 sau đó tăng dần lên theo cấp số cộng 1, thử nghiệm cho tới khi xe có thể bắt được tất cả các trường hợp line của map đã tạo. Và sau các lần thử với các mốc P khác nhau, nhóm đã chọn ra được P = 17 là phù hợp nhất. Để tối ưu nhất trong việc tìm hệ số, nhóm tiếp tục chia nhỏ khoảng với cấp số cộng 0.1 trong khoảng [16, 18]. Nhưng kết quả trả về tối ưu nhất vẫn là P = 17. Quy trình chọn hệ số D cũng tương tự như vậy.

- Sau khi chọn P, với tính chất của P càng cao thì tốc độ tìm về set point càng nhanh nhưng cùng với đó là độ vọt lố càng lớn khiến xe lắc qua lắc lại mạnh. Để khắc phục điều này, nhóm đến với bước tiếp theo là chọn hệ số D. Hệ số D trong thuật toán PID là thành phần tạo ra phản hồi tỉ lệ với tốc độ thay đổi của sai số, tức là tác động đến sự biến đổi của sai số theo thời gian. Điều này giúp hãm lại độ vọt lố của P gây ra, triệt tiêu đi những sai số mà P gây ra trong quá trình chuyển động. Nếu chọn D quá thấp, sẽ không thể triệt tiêu được nhiều độ vọt lố của P gây ra, khiến chuyển động vẫn bị lắc qua lắc lại nhiều, kém ổn định. Nếu chọn D quá cao, sẽ khiến chuyển động của xe quá nhạy cảm với biến động của sai số, khiến xe trì lại nhiều, giảm độ mượt của chuyển động và hiệu suất của xe. Nhóm cũng thực hiện chọn hệ số D giống với quy trình chọn hệ số P. Và cuối cùng nhóm chọn được hệ số D = 3 là phù hợp nhất với xe, giảm

được đáng kể độ lắc, giúp xe chuyển động bám line hơn và xe không bị trì lại khi chuyển động.

- Cuối cùng là hệ số I, do tính chất của xe sẽ lắc qua lắc lại thường xuyên dù đã có tối ưu của hai hệ số P và D, nhưng đây không phải là sai số tĩnh, không phải là steady state error nên sử dụng hệ số I là không phù hợp. Nên nhóm đã quyết định chọn I = 0.

- Nhóm kết luận ba hệ số PID phù hợp nhất cho xe mà nhóm chọn được sẽ là: `P = 17, I = 0, D = 3.`

5.9.2. Xử lý ngã rẽ và các trường hợp đặc biệt chữ T, dấu cộng, điểm đích

```
int bitSensor = 0;
  for(int i = 0; i < 8; i++)
  {
    bitSensor = (bitSensor << 1) | sensorD[i];  // Chuyển mảng sensorD[] thành bit
  }

  switch(bitSensor)
  {
  // left side detection
    case 0b11110000: direction = "left"; break;
    case 0b11111000: direction = "left"; break;
    case 0b11111100: direction = "left"; break;
    case 0b11111110: direction = "left"; break;

  // right side detection
    case 0b00001111: direction = "right"; break;
    case 0b00011111: direction = "right"; break;
    case 0b00111111: direction = "right"; break;
    case 0b01111111: direction = "right"; break;
  }

//Rẽ khúc cua gấp
  if(bitSensor == 0)
  {
```

```
    if (direction != "straight")
    {
      if(direction == "right")
      {
        turnRight();
      }
      else
      {
        turnLeft();
      }
    }
  }

//Chữ T
  else if(bitSensor == 255)
  {
    move_forward(move_forward_time);
    read_black_line();

    if (sumSensor == 0)
    {
      turnLeft();
    }
//Dấu cộng
    else
    {
      if(bitSensor > 0 && bitSensor < 255)
      {
        direction == "straight";
      }
    }
//Đích
    while(sumSensor == 8)
    {
      stop();
    }
  }
}
```

- **Hoạt động:**
  - Khai báo giá trị của `bitSensor = 0` để reset giá trị này.
  - Lệnh được thực thi trong vòng lặp for ngay sau biến `bitSensor` vừa được khai báo dùng để chuyển đổi tín hiệu nhị phân của mảng `sensorD[i]` sang bit. Ví dụ: mảng `sensorD[i]` trả về 11111111 có nghĩa `bitSensor = 255.`
  - `Switch – case bitSensor` dùng để liệt kê ra những trường hợp rẽ trái phải tại các góc nhọn và góc 90 độ của xe. default của lệnh `switch case` đã được ở những dòng code đầu tiên `direction = "straight"`, có nghĩa là đi thẳng. Các trường hợp không có trong `switch-case` đều trả về `straight.`

```
03:22:01.348 -> right  0 0 0 1 1 1 1 1
03:22:01.382 -> right  0 0 0 1 1 1 1 1
03:22:01.382 -> right  0 0 0 1 1 1 1 1
03:22:01.415 -> right  0 0 0 1 1 1 1 1
03:22:01.449 -> right  0 0 0 1 1 1 1 1
```

Hình 48: Vị trí của xe khi chuẩn bị rẽ phải

```
03:23:33.562 -> left  1 1 1 1 1 0 0 0
03:23:33.596 -> left  1 1 1 1 1 0 0 0
03:23:33.629 -> left  1 1 1 1 1 0 0 0
03:23:33.662 -> left  1 1 1 1 1 0 0 0
03:23:33.662 -> left  1 1 1 1 1 0 0 0
```

Hình 49: Vị trí của xe khi chuẩn bị rẽ trái

```
03:24:44.831 -> straight  0 0 0 1 1 0 0 0
03:24:44.831 -> straight  0 0 0 1 1 0 0 0
03:24:44.864 -> straight  0 0 0 1 1 0 0 0
03:24:44.897 -> straight  0 0 0 1 1 0 0 0
03:24:44.930 -> straight  0 0 0 1 1 0 0 0
```

Hình 50: Vị trí của xe khi đi thẳng

- Trong logic rẽ trái phải của xe được chạy theo luồng như sau:

Code:

```
//Rẽ khúc cua gấp
  if(bitSensor == 0)
  {
    if (direction != "straight")
    {
      if(direction == "right")
      {
        turnRight();
      }
      else
      {
        turnLeft();
      }
    }
  }
```

- Điều kiện đầu tiên `if(bitSensor == 0)` là để kiểm tra xem xe đã vọt qua line đen ở ngã rẽ chưa. Phải kiểm tra điều kiện này vì khi xe đi nhanh đến điểm cua, nó sẽ lao lên một chút theo quán tính của xe, tại tới điểm cuối cùng trước khi rẽ line sensor sẽ nằm hoàn toàn ở ngoài line.
- Nếu điều kiện đầu tiên đúng, sẽ đến tới điều kiện thứ hai là `if(direction != "straight"),` điều này để kiểm tra rằng xe có đang ở hướng thẳng hay không, khi đúng vào trường hợp rẽ trái hoặc phải, thì `direction` đã được lưu là "`left`" hoặc "`right`" rồi, khi này điều kiện thoã mãn không phải "`straight`".
- Nếu điều kiện `if(direction != "straight")` đúng thì sẽ tới điều kiện tiếp là `if(direction == "right")` thì nó sẽ thực thi thi hàm `turnRight()` để rẽ phải và thêm một câu điều kiện `else` nữa, do ở trên đã thoả mãn điều kiện direction khác "`straight`" rồi, nên else

còn lại chắc chắn `direction == "left”` và sẽ thực thi hàm `turnLeft()` để rẽ trái.

- Trong logic chuyển động tại các trường hợp đặc biệt của xe được chạy theo luồng như sau:

Code:

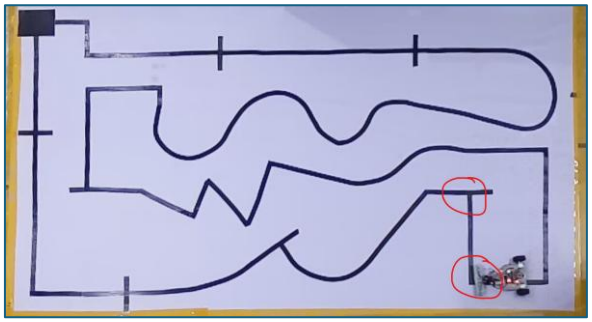
```
//Chữ T
  else if(bitSensor == 255)
  {
    move_forward(move_forward_time);
    read_black_line();

    if (sumSensor == 0)
    {
      turnLeft();
    }
//Dấu cộng
    else
    {
      if(bitSensor > 0 && bitSensor < 255)
      {
        direction == "straight";
      }
    }
//Đích
    while(sumSensor == 8)
    {
      stop();
    }
  }
```

- Tiếp tới là một điều kiện tiếp theo sau `if(bitSensor == 0)` là `else if(bitSensor == 255)`. Điều kiện này dùng để xét tới các trường hợp cả 8 cảm biến trên line sensor cùng trên line đen. Đó là

chữ T, dấu cộng và điểm đích. Hàm được sử dụng ngay sau đó là `move_forward(move_forward_time)`, có tác dụng tịnh tiến thêm một đoạn trong `move_forward_time = 60ms` đã được khai báo ở phần đầu tiên của code, với tốc độ xe hiện tại thì thời gian này đủ để xe chạy qua line đen, hàm này có tác dụng để phân biệt giữa 3 trường hợp chữ T, dấu cộng và điểm đích. Sau đó thêm hàm `read_black_line()` để cập nhật tiếp thông tin của line sensor

- **Chữ T**: Khi cả 8 cảm biến cùng chạm tới line đen xe sẽ đi thêm 60ms nữa, sau đó nó sẽ kiểm tra xem cảm biến trả về giá trị như nào, với trường hợp của chữ T khi đi vọt thêm 60ms, hàm `read_black_line()` sẽ không còn phát hiện được line nào nữa nên điều kiện xét cho trường hợp này là `if(sumSensor == 0)`, nếu điều kiện đúng sẽ thực thi hàm `turnLeft()`. Trong trường hợp này chỉ có thể tuỳ thuộc vào map để cho điều kiện này thực thi `turnLeft()` hoặc `turnRight()` vì xe hiện tại không thể biết được đường đi tiếp theo sau khi rẽ sẽ ở bên trái hay ở bên phải. Nếu đường đi có cả ở 2 bên thì không có vấn đề gì, để `turnLeft()` hay `turnRight()` đều được.
    - Lý do sử dụng `sumSensor == 0` thay vì `bitSensor == 0` là do trên map trước khi tới đoạn rẽ chữ T, xe đã thực hiện 1 lệnh rẽ phải `turnRight()` mà tại thời điểm này direction đã được lưu thành "right" cho trường hợp `if(bitSensor == 0)` rồi, vì vậy nếu trường hợp chữ T mà cũng sử dụng `if(bitSensor == 0)` thì nó sẽ rẽ phải thay vì rẽ trái như đã định.



Hình 51: Xe tại vị trí rẽ phải và chuẩn bị tới chữ T

- **Dấu cộng**: Tương tự trường hợp chữ T, với trường hợp dấu cộng, nếu đã thực hiện `move_forward()` mà cảm biến biến nằm trong khoảng `(0 < bitSensor < 255)`, tức là có ít nhất 1 cảm biến phát hiện ra line đen, nên điều kiện xét của trường hợp này là `if(bitSensor > 0 && bitSensor < 255)`. Nếu điều kiện này đúng, sẽ được set `direction == "straight"`, làm cho xe tiếp tục đi thẳng khi đi qua dấu cộng.
- **Điểm đích**: Cuối cùng đến với trường hợp điểm đích, sau khi đã thực thi hàm `move_forward()`, cả 8 cảm biến của line sensor vẫn nhận diện được line đen, nên trường hợp này sẽ có 1 vòng lặp `while(sumSensor == 8)` để luôn dừng khi trường hợp này xảy ra, và để dừng xe thì dùng hàm `stop()` trong vòng lặp while.
- Nền trắng: Đây là trường hợp sẽ hiếm khi xảy ra tại đường đi trong map tại hầu hết các trường hợp của xe dò line sẽ là vòng kín. Nó chỉ xảy ra tại thời điểm thử nghiệm xe khi phía cuối đường của line ra thẳng nền trắng. Lúc này xe sẽ có xu hướng đi lệch sang bên trái. Khi này `sumSensor = 0`, `sumWeight = 0` và `currPos = sumWeight / sumSensor = -1`, tất cả các biến trên đều được khai báo theo kiểu int.

  Và sau các bước tính toán thì `leftSpeed`, `rightSpeed` có kết quả theo hình dưới đây, với kết quả này sẽ khiến cho xe đi lệnh sang bên trái.

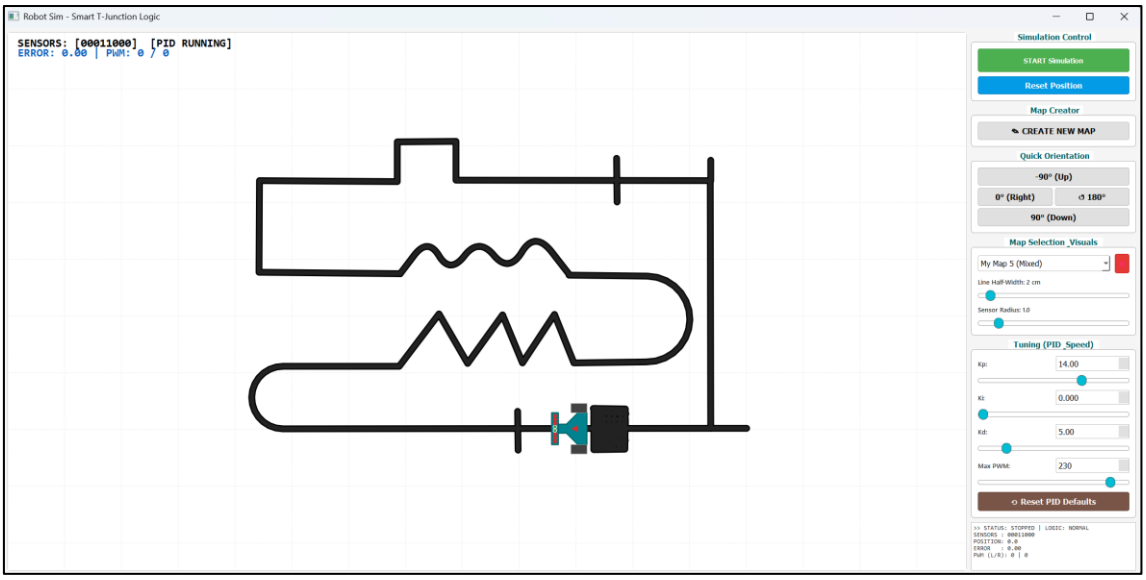
```
21:46:50.314 -> currPos: -1  leftSpeed: -527  leftSpeed: 1037  0 0 0 0 0 0 0 0
21:46:50.381 -> currPos: -1  leftSpeed: -527  leftSpeed: 1037  0 0 0 0 0 0 0 0
21:46:50.448 -> currPos: -1  leftSpeed: -527  leftSpeed: 1037  0 0 0 0 0 0 0 0
21:46:50.515 -> currPos: -1  leftSpeed: -527  leftSpeed: 1037  0 0 0 0 0 0 0 0
21:46:50.582 -> currPos: -1  leftSpeed: -527  leftSpeed: 1037  0 0 0 0 0 0 0 0
```

Hình 52: Khi 8 cảm biến của line sensor ở nền trắng

6. MÔ PHỎNG VÀ TỐI ƯU HÓA THUẬT TOÁN TRÊN PYTHON

6.1. Lý do xây dựng môi trường mô phỏng

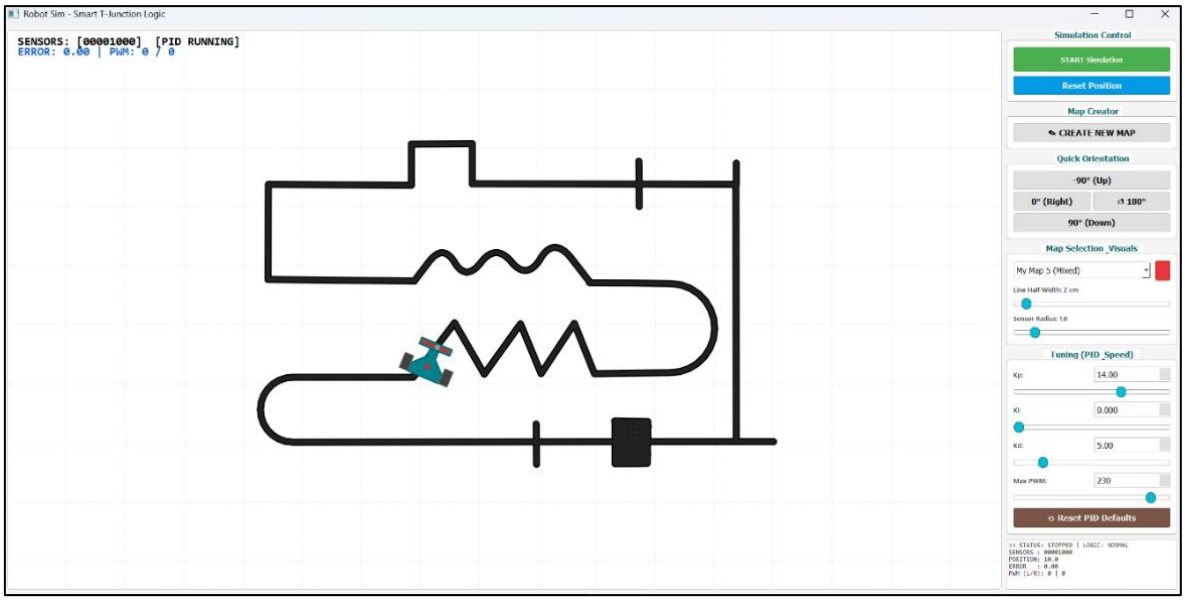
Xây dựng môi trường mô phỏng chủ yếu nhằm hỗ trợ quá trình thử nghiệm và điều chỉnh tham số PID (Kp, Ki, Kd) cho robot dò line một cách nhanh chóng và hiệu quả. Mô phỏng cho phép chạy xe ảo trên các đường đua phức tạp (đường cong, zigzag, ngã rẽ) ngay trên máy tính. Người dùng có thể quan sát tức thì hành vi robot, theo dõi giá trị error, PWM và điều chỉnh. Điều này đặc biệt quan trọng trong giai đoạn phát triển ban đầu, giúp tiết kiệm đáng kể thời gian, chi phí linh kiện và công sức trước khi chuyển sang triển khai thực tế.



Hình 53: Giao diện phần mềm mô phỏng

6.2. Công dụng và hiệu quả thực tiễn

Người dùng có thể thử nghiệm hàng trăm bộ tham số PID chỉ trong vài phút, quan sát hiện tượng rung lắc hoặc lệch line, từ đó chọn ra bộ tham số tối ưu nhất.

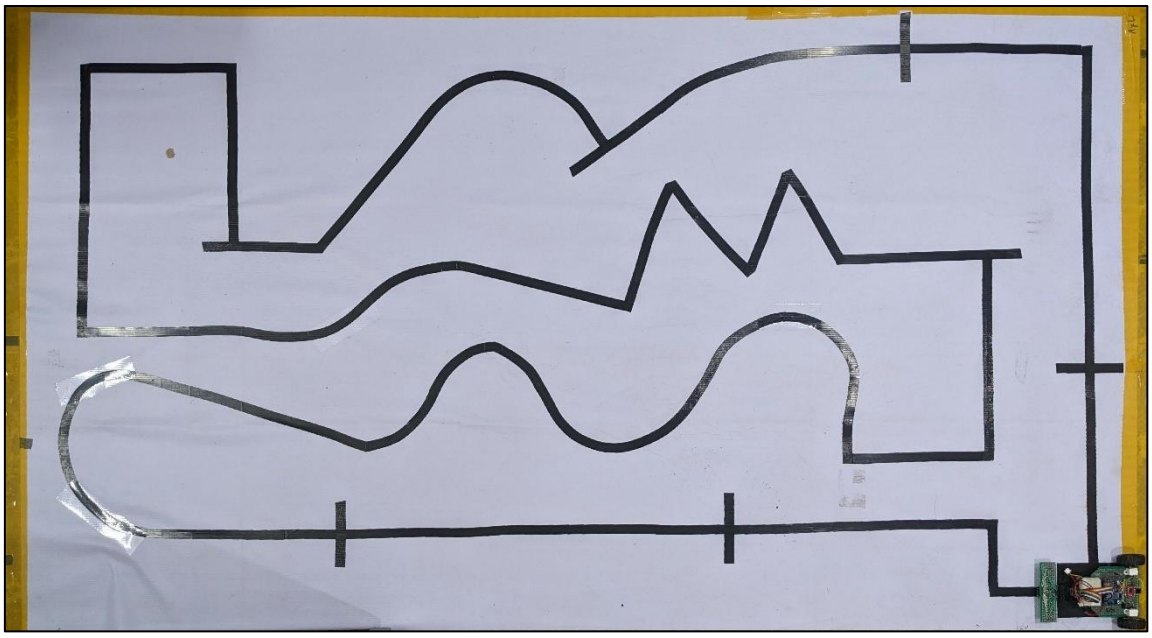


Hình 54: Giao diện phần mềm khi xe chạy

6.3. Kết quả mô phỏng và đối chiếu với xe thật

Trong môi trường mô phỏng, robot hoạt động rất ổn định: error gần bằng 0, PWM cân bằng giữa hai bánh, xe bám line chính xác qua các đoạn zigzag và đường cong phức tạp mà không bị lệch đáng kể, ngay cả ở tốc độ cao. Thuật toán PID sau khi được tuning cho phản ứng nhanh, ít overshoot và không rung lắc, giúp xe hoàn thành toàn bộ đường đua một cách mượt mà. Khi đối chiếu với xe thật, kết quả cho thấy độ tương đồng cao khoảng 85-90%. Xe thực tế sau khi nạp cùng bộ tham số PID từ mô phỏng đã bám line tốt, giảm thiểu rung lắc ở đoạn cong và duy trì tốc độ ổn định tương tự môi trường ảo. Tuy nhiên, vẫn tồn tại một số chênh lệch nhỏ do các yếu tố thực tế như ma sát bánh xe, độ chính xác cảm biến hồng ngoại bị ảnh hưởng bởi ánh sáng môi trường, hoặc độ lệch cơ khí của khung xe. Sau khi tinh chỉnh nhẹ thêm dựa trên quan sát thực tế (thường chỉ tăng nhẹ Kp hoặc Kd), xe thật đạt hiệu suất gần như tương đương mô phỏng: hoàn thành đường chạy nhanh, chính xác và ổn định. Kết quả này chứng minh môi trường mô phỏng là công cụ cực kỳ hiệu quả để phát triển và tối ưu robot dò line trước khi triển khai hoàn chỉnh.

7. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ



Hình 55: Hình ảnh sản phẩm xe thực hiện các chức năng trong thực tế.

Mục tiêu đạt được:

- Từ ý tưởng và các chức năng đề ra, nhóm đã hoàn thiện sản phẩm đúng với hướng phát triển ý tưởng của nhóm, xe có chức năng nhận biết đường line và di chuyển bám theo line, tích hợp được hai chức năng trong một sản phẩm
- Nhóm đã hoàn thành được việc thiết kế, lắp ráp và hoàn thiện robot dò line tự hành có khả năng bám đường line và ổn định trong quá trình di chuyển
- Thực nghiệm và triển khai tối ưu được bộ điều khiển phản hồi, làm robot có thể xử lý tốt trong các tình huống trên đường line.

Hiệu quả:

- Xe chạy ổn định khi có thuật toán giúp hệ thống dự đoán được sự thay đổi của sai số đặc biệt ở các khúc cua giúp xe không bị trượt ra khỏi đường line
- Xe có thể chạy trong môi trường thiếu sáng.

- Xe có thể giải quyết mâu thuẫn giữa tốc độ và sự cân bằng, điều phối vận tốc dựa trên mức độ lệch ra khỏi đường line giúp line luôn ở trọng tâm của xe. Tám cảm biến của xe giúp xe định vị được vị trí từ đó có thể làm cho xe hoạt động êm ái, giảm rung lắc khi chạy trên line

Hạn chế:

- Trong môi trường có nhiều ánh sáng như mặt trời hay trong môi trường không lý tưởng ảnh hưởng đến cảm biến và vạch kẻ làm cho xe có khả năng bị nhiễu, việc cố tối ưu làm xe chạy ổn định cũng làm xe khó trong việc xử lý triệt để các sai số.
- Bề mặt sàn cũng ảnh hưởng rất nhiều tới việc nhận diện của cảm biến và di chuyển của xe. Xe chỉ hoạt động tốt với bề mặt đường phẳng, ma sát tốt, phần line nên để là màu đen, nền trắng, không bị tác động làm bề mặt bị bẩn hay thay đổi kết cấu,….

8. ĐỊNH HƯỚNG PHÁT TRIỂN

Định hướng cải tiến

- Thuật toán điều khiển thông minh
- Tối ưu hóa PID, thay vì cài đặt sẵn các hệ số, nhóm phát triển thuật toán tự tìm hệ số tối ưu dựa trên độ cong của các khúc cua khi chạy trên đường line
- Sử dụng thêm cảm biến phía trước giúp robot nhìn được khúc cua sớm, từ đó chủ động giảm tốc trước khi cua làm giảm thiểu rủi do về sai số khi chạy

Nâng cấp hệ thống cảm biến:

- Sử dụng camera AI thay thế cho mảng tám cảm biến thu/phát hồng ngoại giúp robot không chỉ dò line mà còn có thể nhận diện được màu sắc vạch kẻ hoặc vật cản từ xa

- Phát triển tính năng tự động hiệu chuẩn để robot xác định ngưỡng trắng / đen dựa trên ánh sáng thực tế, có thể xoay, di chuyển ziczac khi mất dấu line để dò lại line

Phát triển phần cứng và mở rộng quy mô:

- Thay thế động cơ N20 thường bằng động cơ có tích hợp encoder giúp kiểm soát được chính xác tốc độ thực tế của robot làm xe chạy thẳng và tính toán được quãng đường đã đi.
- Thêm màn hình điều khiển để thay đổi tốc độ di chuyển, xuất dữ liệu về vị trí thực tế về máy chủ để từ đó mà điều chỉnh trạng thái robot.
- Thiết kế và tích hợp mạch sạc pin Lithium, có đèn LED báo dung lượng pin còn lại của xe, xây dựng hệ thống sạc thông minh để xe có thể tự dò theo đường line về ổ sạc pin khi pin đến ngưỡng cần sạc.
- Mở rộng và ứng dụng được robot thành hệ thống vận chuyển tự động AGV.

9. TÀI LIỆU THAM KHẢO

[1] Arduino. (n.d.). *Arduino nano*. Arduino.

https://docs.arduino.cc/hardware/nano/

[2] Berl, G. (2019, February 16). PID line follower tuning. Robot Research Lab.

https://robotresearchlab.com/2019/02/16/pid-line-follower-tuning/

[3] Kevin Lynch. (2015, August 12). Empirical PID gain tuning [Video]. YouTube.

https://youtu.be/uXnDwojRb1g

[4] Pololu Corporation. (2001–2019). Arduino library for the Pololu QTR reflectance sensors (Version 3.1.0).

https://www.pololu.com/docs/0j19/all

[5] Texas Instruments. (2023). LM2596 SIMPLE SWITCHER® power converter 150-kHz 3-A step-down voltage regulator (SNVS124G).

https://hshop.vn/mach-giamop-dc-lm2596-3a

[6] ThinkRobotics. (n.d.). PID tuning for line follower robot: Complete how-to guide. ThinkRobotics.

https://thinkrobotics.com/blogs/learn/pid-tuning-for-line-follower-robot-complete-how-to-guide

[7] Toshiba Corporation. (2007). TB6612FNG: Driver IC for dual DC motor (Document No. 070630). Toshiba.

https://cdn.sparkfun.com/datasheets/Robotics/TB6612FNG.pdf